The graph stratification version of the Marcel theorem prover: documentation and examples

Lavinia Randall Holmes

May 16, 2026

Contents

1 Introduction **2**

2 The language of the prover **4**

3 Substitutions **6**

4 The stratification algorithm **7**

5 The sequent prover **9**
5.1 Specific Sequent Rules: Connectives 12
5.2 Axiom 12
5.3 Left rules (Premise rules) 13
5.4 Right rules (Conclusion rules) 13

6 More Sequent Rules **13**
6.1 Rules for Quantifiers 14
6.1.1 Left Rules (Premise Rules) 14
6.1.2 Right Rules (Conclusion Rules) 14
6.1.3 Comments on Quantifier Rules 14
6.2 Rules for Membership 15
6.2.1 Left Rule 15
6.2.2 Right Rule 15
6.3 Rules for Equality 15
6.3.1 Left Rule 15

6.3.2 Right Rule 15
6.4 Global Substitution 16

7 Sample proofs **16**
7.1 Inclusion is transitive 16
7.2 Equality is transitive 37
7.3 Coextensionality implies Leibniz equality 70
7.4 A weird example with unknown variables 101
7.5 Atomic inclusion is membership 109
7.6 First projection theorem for the ordered pair 124

1 Introduction

This document introduces another version of the Marcel theorem prover for stratified set theory, so-called because it implements the system of sequent calculus for SF (stratified set theory with no extensionality axiom but with a set abstraction operator) defined by Marcel Crabbé and shown by him to satisfy cut elimination (the right rule for equality that we use actually gives the full extensionality of NF, and cut elimination is not known for this system, but this can be corrected if desired).

This version differs from earlier versions in having no special commands for particular logical operations. All transformations based on rules for particular logical constructions are handled by `left()`, `right()` or `done()` with assistance from `setunknown` to set particular instances of variables introduced. It differs from other versions in frankly using full extensionality. We know that New Foundations itself is consistent. We could easily install the appropriate modifications of the rules to give SF or NFU. Unlike the previous version, it does stratification itself automatically (the reasons for this, which are actually the initial trigger of this version of the project, are discussed shortly). The just previous version used user supplied type indices on bound variables [which we still regard as a good approach]. This version handles defined notions quite differently from earlier versions, and we are rather interested in the results. This version handles weak stratification issues and variable binding issues generally by keeping track of occurrences of variables invisibly. This means that in fact only bound variables which are connected to the binding variable of a set abstract need to be typed consistently, which gives a stratification criterion weaker than Marcel's weak stratification (in an

inessential way, but it is weaker). Occurrences of variables are only identified if they are bound by the same binder. This has the curious effect that the prover entirely ignores issues of variable capture: apparent variable capture can happen in the display, but the prover knows which occurrences of variables of the same shape are actually bound in a context with a binder of that shape, and currently keeps track of this without telling the user. This doesn't seem to actually happen often, but I probably ought to fix the display for user comfort. The input language is entirely in Polish notation, which we did for ease of prototyping. The output language is more familiar-looking. We probably ought to write a parser for the output language.

We are actively debugging the code: the example transcripts in the back need to be refreshed now and then and they will not always be up to date.

This particular version was immediately inspired by contemplation of the proposal by Ryan Nolan that the Bellman-Ford algorithm for finding shortest paths in weighted graphs in which edges might have negative weight could be used for stratification testing. Our actual proposal owes nothing in its details to Nolan because the use of that algorithm, while it works in principle, is a bad idea. We present a rather different algorithm to be applied to the graph on variables in the directed graph determined by the atomic formulas in a graph with weights determined by relative type and order of the variables, similar to Dijkstra's algorithm. The use of Bellman-Ford is a bad idea because the execution of this algorithm can be aborted the second time that any edge is "relaxed" (or if an overestimate of the distance is seen to exist, something that Bellman-Ford doesnt even notice) with a report of stratification failure.

It is also motivated by the concurrent and quite independent study by Thomas Forster and myself of the properties of the graph on variables in a formula determined by the atomic formulas in the graph (and in particular the properties of the notion of connectedness for formulas determined by connectedness of this graph: I was already thinking about the graph on variables determined by a formula and its relation to stratification when I encountered Nolan's suggestion.

This version also follows the path of assigning invisible occurrence data to every variable in a term or formula. Occurrences of variables are typed, not the variables themselves, and this enables direct implemention of weak stratification (in fact, even weaker than in Marcel's system: all that is required is that variables connected to the binding variable in each set abstract have consistent types in relation to the binding variable.) This has (at least at this

initial stage) the perhaps perverse feature that the definition of substitution can ignore variable capture: the reason is that the only instance in which occurrences of variables are regarded as identical is when they are bound by the same quantifier or set abstractor. When substitutions are carried out, the substitution term has all of its occurrence information displaced to a new range (so that no identification can exist between any variable in the substituted term and any binder into whose scope it is substituted). Attention to the moral law probably requires us to install a warning to the viewer when free variables are introduced into scopes of binders using variables of the same shape: the distinction is not currently apparent to the viewer in term or formula display. But the prover knows and does not make errors due to apparent variable capture.

2 The language of the prover

For rapid prototyping, we wrote a parser using Polish notation, but the display function will look more familiar.

Terms are variables (letters `a-z` with one or more primes (`‘` or `*`) affixed) or set abstracts (discussed subsequently). The primes are prefix in the entry notation (to support the Polish nature of the parser) and postfix in the display notation. Variables have invisible occurrence indices: a new index is generated for each new variable, but then each occurrence of a variable bound by a quantifier or set abstractor is coerced to have the same occurrence index as the binding variable. Occurrence indices are also manipulated by substitution and by the process of generating fresh variables in ways to be discussed below.

A set abstract in the input notation takes the form {`xf`, { being the abstraction operator, `x` being a variable, and `f` being a formula. In the display notation it takes the more familiar form $\{x \mid \phi\}$, ϕ being the same formula but in display notation.

Formulas are of various flavors we now list.

Atomic formulas are of the forms `=tu` (in display form $T = U$) and `etu` (display form $T\, e\, U$ in the prover: we might write $T \in U$) where `t` and `u` are terms with display forms T and U respectively. These are atomic assertions of equality and membership.

Boolean formulas are of the forms `Cfg` where C is a connective, one of `&,V,>,X` in the input language. Formulas `&fg, Vfg, >fg, Xfg` take the

more familiar forms $(\phi \,\&\, \psi)$, $(\phi \, V \, \psi)$, $(\phi -> \psi)$ and $(\phi == \psi)$ respectively, where `f` and `g` are formulas with display forms ϕ and ψ. We may choose to write $(\phi \wedge \psi)$, $(\phi \vee \psi)$, $(\phi \rightarrow \psi)$ and $(\phi \leftrightarrow \psi)$ for these familiar logical operations.

In a separate category due to its arity is the negation $\sim$`f` which is written $\sim \phi$ in display, where `f` is a formula with display form ϕ. We may also write $\neg\phi$.

Quantified formulas are of the forms `Axf`, `Exf` which are written $(Ax : \phi)$ and $(Ex : \phi)$ in display (where `f` is a formula with display forms ϕ)). We may also write the usual $(\forall x : \phi)$ and $(\exists x : \phi)$ here.

The `testt` and `testf` formulas can be used to practice term and formula entry.

```
testt("{x=xx")
'{x | x = x}'
testf("&exyeyz")
'(x e y & y e z)'
```

The user should be warned that the parser does stratification checks on set abstracts and will not allow set abstracts which are not weakly stratified in a suitable sense.

```
testt("{xexx")
x e x
x e x
['stratification failure', [['x', 2], -1, ['x', 2]]]
'!!!'
'!!!'
```

Details to be explained later. But notice that the attempt to present $\{x \mid x \in x\}$ is a failure: one never even sees it, though one sees its body.

A new feature which has been introduced but which needs debugging is a feature supporting defined terms and formulas. I did this in a possibly eccentric way which may cause me vast difficulties, or may prove really convenient. I am not sure.

Capital letters, possibly primed, may be used to represent defined terms and formulas. The defined terms and formulas are introduced with `deft` and `deff` commands to be documented. They are further used by supplying values for variables in the defined term or formula. For example, if we define `I` as $\{x : x = a\}$ then we can abbreviate the singleton of t as $I(a : t)$ (`:atI` in the input

language) [$I(a : x)$ actually works correctly: the reason is that variables derived from definition text are prefixed with at signs (`@`), preventing apparent variable capture]. If we define `C` as $(\forall x : x \in a \to b \in a)$ then we can represent $t \subseteq u$ as $C(a : t)(b : u)$ (which is `:bu:atC` in the input language; exchanging order of the arguments would be harmless, as the variable name controls where the argument is put). Stratification for these notations is defined (but will no doubt require some debugging): the term or formula in a definition must be stratified in the strong sense, so each variable in it has a fixed type, and this information can be used to build a stratification graph for a term with defined notions in it without unpacking the definitions (the table of relative types of variables in the body of a definition is stored with the definition).

Notice that defined notions may have arbitrary numbers of arguments which may be supplied in any order (since they have associated keys). The use of defined terms is a bit weird from the standpoint of variable use. It is best to think of the defined term and the keys for arguments as bits of text. One cannot substitute for them and the keys are not variables in the expression. When a definition with an argument is evaluated, all unused variables get prefixed with an at sign: this avoids not actual error but a serious risk of apparent variable capture. It also warns you when not all arguments are used.

Merits of this: there are just two forms of definition, and no more are needed. Arguments can be supplied in any order, and any number of arguments is supported. Inhomogeneous operations are supported (arguments can have arbitrary relative type, though defined terms and formulas must be stratified in the strong sense). Demerits: the notation is weird. The order of the arguments gets reversed between the input and output notation, though this is semantically harmless.

Defined constant terms are permitted (defined terms do not require arguments); defined constant formulas are not permitted (a defined formula must appear with at least one argument). In a bit of syntactic sugar, if the first two keys supplied to a defined notation are `a` and `b`, it is displayed in infix form (this works both for terms and for formulas).

3 Substitutions

There are two kinds of substitution. One replaces all free instances of a particular occurrence of a variable in a context – this is used for operations on binding contexts (quantifier rules and comprehension). The other replaces all free variables with the same surface form (ignoring the occurrence index) in a context.

In both kinds of substitution, the term substituted in has all of its occurrence indices displaced above all existing text, so variables actually cannot be captured

when substituted into binding scopes...but the display doesn't show this [something we ought to fix]. Later (after introducing the sequent prover) I'll put in an example of the weirdness of apparent variable capture.

4 The stratification algorithm

The first step of the stratification checker is to build a directed graph from the formula to be checked. Each atomic formula generates edges in both directions, with weight equal to the difference in relative type. $x = y$ creates edges with weight 0 from x to y and from y to x. $x \in y$ creates an edge from x to y with weight 1 and an edge from y to x with weight -1. Edges are also created by atomic formulas involving set abstracts. For example, $\{x \mid \phi\} \in z$ creates an edge from x to z of weight 2 and an edge from z to x of length -2, as well as an entire graph from ϕ.

Notice that distinct occurrences of free variables are distinct nodes in the graph. $x \in x$ is a (weakly) stratified formula, with the two occurrences of x having different relative types. But occurrences of bound variables with the same surface form and the same binder are identified: $\{x \mid x \in x\}$ raises a stratification error.

The stratification algorithm takes as input a node in the weighted graph and the entire graph. It computes distances from the selected node to all nodes connected with it. It does this by starting with a very rough estimate of the distance function (distance 0 from the selected node to itself [and the trivial path from the selected node to itself as an extra bit of data] and infinite distance to all other nodes) and proceeds through the nodes of the graph improving the estimate at each step. When a node is encountered, if there is no finite distance from the selected node already computed, we move it to the end of the list and proceed to the next node. If there is a finite distance from the selected node to the current node, we compute new estimates of the distance from the selected node to each neighbor of the current node. If the distance to a neighbor is infinite, we get a finite estimate, and add that information and a path from the selected node to the neighbor [both computed in the obvious way] to the data for that neighboring node. If the distance to a neighbor is finite, and our new estimate of the distance is the same, we do nothing to that node. If the distance to a neighbor is finite and our new estimate is different from the original estimate (either an overestimate or an underestimate), we can stop the entire process and report stratification failure, with supporting data consisting of a cycle of non-zero length constructed from the path to the current node, the path to the neighboring node previously computed, and the edge between them. [This is why using the Bellman-Ford algorithm does not make sense: it will work, but the very first time that a distance is adjusted

from a finite value to a smaller finite value, all subsequent work is a waste: and the Bellman-Ford algorithm entirely ignores overestimates, which are also evidence of failure.] When we arrive at the end of the list of vertices, we will have computed all finite distances from the selected node to other nodes, if no stratification error has been found, and we can return the partial distance function, which is in effect a relative type assignment witnessing stratification. Nodes not connected to the selected node by a path are not assigned distances.

The system actually uses the stratification algorithm only on set abstracts [it is also used on definition texts, which must be stratified in the strict sense], ensuring that types can be assigned to each bound variable connected to the binding variable by a path in the graph in a consistent way. This is even weaker than the usual "weak stratification": for example
$\{x \mid x = x \wedge (\exists y : y \notin y)\}$ is stratified for our purposes.

```
testt("{x&=xxAy~eyy")
'{x | (x = x & (Ay : ~y e y))}'
```

The function `strattest` can be used to check a formula (not a term) for stratification.

```
testf("e{x=xx{x=xx")
'{x | x = x} e {x | x = x}'

strattest("e{x=xx{x=xx")
[[['x', 27], [1], [['x', 23], 1, ['x', 27]]], [['x', 23], [0],
[['x', 23]]]]
```

In this formula, there are two different occurrences of x, and the stratification data supplied gives distances for each of them and a path from the selected node (chosen automatically by the testing function) to each of the two nodes.

```
testf("eu{x&exy&eyzezx")
'u e {x | (x e y & (y e z & z e x))}'
strattest ("eu{x&exy&eyzezx")
[[['z', 52], [-1], [['u', 46], 0, ['x', 47], -1, ['z', 52]]],
[['y', 49], [1], [['u', 46], 0, ['x', 47], 1, ['y', 49]]],
```

```
[['x', 47], [0], [['u', 46], 0, ['x', 47]]], [['u', 46], [0],
[['u', 46]]], [['y', 50], [], []], [['z', 51], [], []]]

testf("eu{xEyEz&exy&eyzezx")
(Ey : (Ez : (x e y & (y e z & z e x))))
['stratification failure', [['x', 6], 1, ['y', 7], 1, ['z', 8], 1, ['x', 6]]]
'???'
```

Here we have two related examples. Neither are stratified in the strict sense, but the first is weakly stratified, because y and z are free, so different occurrences can be assigned different types.

In the second formula, where we quantify over y and z, there is no need to run the stratification checker: just trying to display the formula with the unstratified set abstract in it is a sufficient disaster to cause the checker to complain.

To assist with reading the output: a successful stratification is a list of triples, a node followed by the singleton list of the distance to the selected node (or the empty list to signify infinite distance) followed by a path (decorated with weights of edges) from the selected node to the given node. A report of failure contains two paths from the selected node and an edge between the termini of those two paths, witnessing a change in estimated finite distance.

The emphasis of this work has not been on creating nifty displays for stratification information: I could improve this, but have not done so so far.

I advise the reader that stratification data is now more crowded. Installing the definition feature required that I create graphs for terms as well as formulas, and graphs for terms need a base node for the entire term. So all subterms acquire base nodes, and there are a lot of additional nodes in the graphs (such nodes have names which are numerals). I clean the base nodes out of stratification display output, but I cannot remove them from graphs [at least, I do not see a way to do it]; they are essential to making the definition feature work.

We note that there is now also a tester for stratified terms rather than formulas (`strattest2`)

5 The sequent prover

The remainder of the work is development of a user driven sequent prover for NF (which could be adapted to a prover for SF [for which the cut elimination result exists] or NFU by changing the right equality rule). This parallels my earlier work on versions of the Marcel prover, but it does have interesting features.

The general plan of the sequent prover is that one uses `start s` to introduce a sequent of the form

$$\frac{}{P}$$

where P is the result of parsing the string s.
One then has a limited number of commands as options.

- The `right()` command applies an appropriate right rule to the first proposition on the right (below the line that is)
- The `left()` command applies an appropriate left rule to the first proposition on the left (above the line, that is).
- The `getleft(n)` command brings the nth proposition on the left (above the horizontal bar, actually) to the front.
- The `getright()` command brings the n proposition on the right to the front.
- The `done()` command invites the prover to recognize that the current sequent is an axiom (that the propositions at the beginnings of the lists on left and right are the same). Identity of propositions up to α-equivalence, ignoring occurrence data, is supported.

  The `done()` command is dynamic: its underlying equality test will make an equation between an unknown variable in one term or formula and the matching term in the other term or formula true by executing the `setunknown` command on the fly. It does this safely (it tests that the `setunknown` command will execute without error). A static equality test will probably be needed eventually, but `done` is currently its only client.

  A further, even more dangerous feature has been added. When the equality function checks whether $x \in u?$ matches $P(x)$ (where $u?$ is an unknown and $P(x)$ is any stratified formula in x, it will attempt to set the unknown $u?$ to the value $\{x \mid P(x)\}$. It does (mod bugs) check that this can actually be done. This is a kind of higher order matching. It can be very nice for forestalling the need to manually enter values to be assigned to unknowns.

  I have also added reflexivity of equality as an axiom, and the `done()` command handles this. This may have dynamic effects if the prover can make the terms in an equation equal by setting unknowns suitably. The other rules prove reflexivity of equality, but this should remove lots of junk lines from proofs.

Where the `done` command fails, one might want to use `back()` to restore the previous state, because the prover might have made undesired global substitutions for unknowns.

- The `setunknown(u,t)` command allows one to set the value (globally in the proof) of an "unknown" `u` (a free variable introduced by the right rule for the existential quantifier or the left rule for the universal quantifier) with a term `t`. In a usual presentation, a specific term is supplied when the rule is applied: here the unity of the `left()` and `right` rules is preserved by separating out the choice of witness. It may also be useful to delay choosing the witness until it is evident from developments in the proof what might work. There are limitations on what can replace an "unknown" [the term replacing it cannot contain any fresh variable introduced by quantifier rules which is introduced after the unknown which is being replaced].

- The `Cut(f)` command executes the cut rule with the formula `f`.

- The `deft(key,term)` command introduces a defined term (defining `key`, a possibly primed capital letter, as `term`) and the `deff(key,formula)` command similarly introduces a defined formula. These can further be accessed by assigning values to variables: in the definiendum: a defined term can be used as a constant, but a defined formula must have at least one assignment. I need to provide examples (they are present in the last section).

- The `savetheorem` command saves the current theorem: it takes the name to be given to the theorem as its sole argument.

- The `theoremcut` command has as its argument the name of a theorem, whose conclusion is introduced as a hypothesis in the current sequent. This allows saved theorems to be used.

- The `look()` command lets you look at the current sequent.

- The `skip()` command lets you move the current sequent to the end of the work queue and go to the next one. This lets you cycle through all the work to be done and view it (This command is much more sensible than its analogues in older versions, where we used an actual tree rather than a list of sequents as our data structure).

- The `back()` command undoes a command, roughly speaking. A stack of proof states is updated from time to time. This function is not entirely reliable [but seems to be improving.]

- The `setlog` command sets the name of a log file to which user commands will be appended as they are executed. This creates scripts which can be used to reconstruct work done under the prover. I close a log file by setting the log file to a default name (`setlog (‘‘done’’)`). There must be a directory called `logs` in the current directory for a log file to be set up successfully.

The prover maintains a stack of sequents needing attention. When the `left()` or `right()` rule is applied, one or two sequents are appended to the end of the proof [which is a list of sequents with pointers (really just lists of line numbers) from sequents to ones justifying them] and one then proceeds to the first sequent on the stack. Other rules affecting proof state have appropriate effects on the stack.

When every line has a justification, the proof is complete.

The meat of the logic supported is in the functions `leftaction` and `rightaction`. which are used to indicate how to construct the one or two sequents from which a current sequent is derived.

We present the rules. I have copied these with some editing from a manual for a long-ago version of Marcel. There is a special additional left rule for membership in a variable which is described at the end.

This version of Marcel frankly has NF as its logic. This could be changed straightforwardly. The logic of this version is classical, but it could very easily be made intuitionistic.

In addition to applying the rules below, the prover also expands defined terms and formulas in a manner which should be documented.

5.1 Specific Sequent Rules: Connectives

We give left rules and right rules (premise and conclusion rules) for the commonly used connectives, excluding converse implication and xor. The notations Γ and Δ represent arbitrary finite sets of propositions. The sentence closest to the turnstile on the right or left is the first sentence in the right or left list in the prover's presentation. The premise on the left is the one which is presented first by the prover when the rule is applied.

The left rule for the biconditional is lazily handled by definitional expansion.

In our experience the rule which it is somewhat difficult to get used to is the left rule for implication, though with a little thought it can be seen to express the rule of *modus ponens*.

5.2 Axiom

$$\Gamma, A \vdash A, \Delta$$

5.3 Left rules (Premise rules)

$$\frac{\Gamma \vdash A, \Delta}{\Gamma, \neg A \vdash \Delta}$$

$$\frac{\Gamma, A, B \vdash \Delta}{\Gamma, A \wedge B \vdash \Delta}$$

$$\frac{\Gamma, A \vdash \Delta \qquad \Gamma, B \vdash \Delta}{\Gamma, A \vee B \vdash \Delta}$$

$$\frac{\Gamma \vdash A, \Delta \qquad \Gamma, B \vdash \Delta}{\Gamma, A \to B \vdash \Delta}$$

$$\frac{\Gamma, A \to B, B \to A \vdash \Delta}{\Gamma, A \leftrightarrow B \vdash \Delta}$$

5.4 Right rules (Conclusion rules)

$$\frac{\Gamma, A \vdash \Delta}{\Gamma \vdash \neg A, \Delta}$$

$$\frac{\Gamma \vdash A, B, \Delta}{\Gamma \vdash A \vee B, \Delta}$$

$$\frac{\Gamma \vdash A, \Delta \qquad \Gamma \vdash B, \Delta}{\Gamma \vdash A \wedge B, \Delta}$$

$$\frac{\Gamma, A \vdash B, \Delta}{\Gamma \vdash A \to B, \Delta}$$

$$\frac{\Gamma, A \vdash B, \Delta \qquad \Gamma, B \vdash A, \Delta}{\Gamma \vdash A \leftrightarrow B, \Delta}$$

6 More Sequent Rules

For conventions on how rules are to be read in general, see the section on sequent rules for connectives above.

In the rules which follow, if ϕ is a formula, $\phi[t/x]$ is taken to represent the result of substituting the term t for the variable x in ϕ.

6.1 Rules for Quantifiers

6.1.1 Left Rules (Premise Rules)

$$\frac{\Gamma, \phi[t/x], (\forall x.\phi) \vdash \Delta}{\Gamma, (\forall x.\phi) \vdash \Delta}$$

where t is any term

$$\frac{\Gamma, \phi[a/x] \vdash \Delta}{\Gamma, (\exists x.\phi) \vdash \Delta}$$

where a is a variable not appearing in the conclusion

6.1.2 Right Rules (Conclusion Rules)

$$\frac{\Gamma \vdash \phi[a/x], \Delta}{\Gamma \vdash (\forall x.\phi), \Delta}$$

where a is a variable not appearing in the conclusion

$$\frac{\Gamma \vdash \phi[t/x], (\exists x.\phi), \Delta}{\Gamma \vdash (\exists x.\phi), \Delta}$$

where t is any term

6.1.3 Comments on Quantifier Rules

In some of the quantifier rules, we have retained the quantified sentence from the conclusion in the premise. This is so that we can avoid formalizing notions of copying and reordering formulas in sequents: a quantified formula may be reused several times in a proof, and if it were erased by the application of the rule we would need to copy it explicitly. Another advantage is that it preserves precise equivalence of the conclusion with the conjunction of all the premises, which is a feature of all the sequent rules of Marcel.

The rules requiring input of a new variable a supply a computer-generated variable. In the original version of this prover, the rules involving a new term t were implemented by separate commands with the term t as a parameter. In the current version, the computer supplies a new "unknown variable" which can subsequently be replaced by a term: the advantage is that the same command can then handle all basic sequent rules. The length and readability of computer generated new variables is greatly improved by having two primes instead of one.

6.2 Rules for Membership

6.2.1 Left Rule

$$\frac{\Gamma, \phi[t/x] \vdash \Delta}{\Gamma, t \in \{x \mid \phi\} \vdash \Delta}$$

when ϕ is stratified (this condition is handled at parse time)

6.2.2 Right Rule

$$\frac{\Gamma \vdash \phi[t/x], \Delta}{\Gamma \vdash t \in \{x \mid \phi\}, \Delta}$$

when ϕ is stratified (this condition is handled at parse time)

6.3 Rules for Equality

6.3.1 Left Rule

$$\frac{\Gamma, (\forall v : t \in v \leftrightarrow u \in v) \vdash \Delta}{\Gamma, t = u \vdash \Delta}$$

This is just definitional expansion. It allows substitution only in stratified contexts, but this is enough for the full strength of substitution as usually presented.

6.3.2 Right Rule

$$\frac{\Gamma \vdash (Ax.x \in t \leftrightarrow x \in u), \Delta}{\Gamma \vdash t = u, \Delta}$$

This gives full extensionality. It can be revised to give SF (use the same Leibniz formulation as in the left rule) or NFU, a bit more elaborate.

To improve the proof theoretic behavior of extensionality, we added a new left rule for membership, tentatively, which expands $x \in y$ to

$$(\forall v : (\forall w : w \in x \leftrightarrow w \in v) \rightarrow v \in y)$$

on the left (when the other rule does not apply). We can report that this allows proof of Leibniz equality from coextensionality without cut. [we revised the exact form of this rule 5/16/2026].

I have also added reflexivity of equality as an axiom. It is provable from the other axioms, but it is convenient not to have to do it over and over.

6.4 Global Substitution

The left rule for the universal quantifier and the right rule for the existential quantifier require input of a specific term t. What the prover actually supplies is an fresh variable identified as an "unknown", annotated with the list of all fresh variables generated before it.

The variables introduced by the other quantifier rules we call "arbitrary" variables. When unknown and arbitrary variables are considered together, I call them fresh variables.

The `setunknown` command allows the user to set the unknown variable to a particular term. There is a restriction on what terms can replace an unknown variable: the term must have been definable at the time that the unknown variable was generated, so it cannot contain any free or unknown variables which were not already declared at the time it was introduced (this data is stored with the unknown variable).

It is worth noting that the display function suffixes arbitrary variables with ! and unknown variables with ?, and it forbids variables of the same shape which are bound from being introduced. The parser does not require or even understand these suffixes.

An advantage of this approach are that it enables one to delay the choice of a witness: the later progress of the proof may make it more evident what witness will work.

7 Sample proofs

The "by lines `[-1]`" annotations have to do with the fact that the lines are not justified at the time they are displayed in a transcript. If I displayed the saved proofs, the list would refer to the numbers of lines justifying the given line, But the displayed proof file would not show the commands used to carry out the proof.

We note here (5/6/2026) that we have just installed the ability to log all user commands to an executable file, so work can be saved and expanded on (and also run again after revisions of the software. All example files are now executable Python files generated by the prover as logs of proofs I carried out. This looks much the same as what I pasted from Python windows in earlier versions, because the logs contain the state of the display at each step in comments.

7.1 Inclusion is transitive

Here is a proof that inclusion is transitive. The text was generated by the new scripting features of the system. In this proof, we used the `LogTheProof()` com-

mand to append the final form of the proof to the script file as a comment.

```
from graph import *

deff ("C", "Ax>exaexb")
start ("AxAyAz > & :ax:byC :ay:bzC :ax:bzC")

"""

Line number 0



----



0.  (Ax : (Ay : (Az : (((x C y) & (y C z)) -> (x C z)))))

by lines [-1]
Next!"""

right()

"""

Line number 1



----



0.  (Ay : (Az : (((x'! C y) & (y C z)) -> (x'! C z))))

by lines [-1]
Next!"""

right()
```

```
"""

Line number 2



----



0.  (Az : (((x'! C y'!) & (y'! C z)) -> (x'! C z)))

by lines [-1]
Next!"""

right()

"""

Line number 3



----



0.  (((x'! C y'!) & (y'! C z'!)) -> (x'! C z'!))

by lines [-1]
Next!"""

right()

"""

Line number 4
```

```
0.  ((x'! C y'!) & (y'! C z'!))
----



0.  (x'! C z'!)

by lines [-1]
Next!"""

right()

"""

Line number 5



0.  ((x'! C y'!) & (y'! C z'!))
----



0.  (A@x : (@x e x'! -> @x e z'!))

by lines [-1]
Next!"""

right()

"""

Line number 6
```

```
0.  ((x'! C y'!) & (y'! C z'!))
----



0.  (x*! e x'! -> x*! e z'!)

by lines [-1]
Next!"""

right()

"""

Line number 7




0.  x*! e x'!

1.  ((x'! C y'!) & (y'! C z'!))
----



0.  x*! e z'!

by lines [-1]
Next!"""

getleft(1)

"""

Line number 7
```

```
0.  ((x'! C y'!) & (y'! C z'!))

1.  x*! e x'!
----



0.  x*! e z'!

by lines [-1]
Next!"""

left()

"""

Line number 8



0.  (x'! C y'!)

1.  (y'! C z'!)

2.  x*! e x'!
----



0.  x*! e z'!

by lines [-1]
Next!"""

left()

"""

Line number 9
```

```
0.  (A@x : (@x e x'! -> @x e y'!))

1.  (y'! C z'!)

2.  x*! e x'!
----



0.  x*! e z'!

by lines [-1]
Next!"""

left()

"""

Line number 10




0.  (x''? e x'! -> x''? e y'!)

1.  (A@x : (@x e x'! -> @x e y'!))

2.  (y'! C z'!)

3.  x*! e x'!
----



0.  x*! e z'!
```

```
by lines [-1]
Next!"""

left()

"""

Line number 11




0.  (A@x : (@x e x'! -> @x e y'!))

1.  (y'! C z'!)

2.  x*! e x'!
----



0.  x''? e x'!

1.  x*! e z'!

by lines [-1]
Next!"""

setunknown ("x''","*x")

"""

Line number 11




0.  (A@x : (@x e x'! -> @x e y'!))

1.  (y'! C z'!)
```

```
2.  x*! e x'!
----



0.  x*! e x'!

1.  x*! e z'!

by lines [-1]
Next!"""

getleft(2)

"""

Line number 11



0.  x*! e x'!

1.  (A@x : (@x e x'! -> @x e y'!))

2.  (y'! C z'!)
----



0.  x*! e x'!

1.  x*! e z'!

by lines [-1]
Next!"""

done()
```

```
"""

Line number 12



0.  x*! e y'!

1.  (A@x : (@x e x'! -> @x e y'!))

2.  (y'! C z'!)

3.  x*! e x'!
----



0.  x*! e z'!

by lines [-1]
Next!"""

getleft(2)

"""

Line number 12



0.  (y'! C z'!)

1.  x*! e x'!

2.  x*! e y'!

3.  (A@x : (@x e x'! -> @x e y'!))
----
```

```
0.  x*! e z'!

by lines [-1]
Next!"""

left()

"""

Line number 13



0.  (A@x : (@x e y'! -> @x e z'!))

1.  x*! e x'!

2.  x*! e y'!

3.  (A@x : (@x e x'! -> @x e y'!))
----



0.  x*! e z'!

by lines [-1]
Next!"""

left()

"""

Line number 14
```

```
0.  (x'*? e y'! -> x'*? e z'!)

1.  (A@x : (@x e y'! -> @x e z'!))

2.  x*! e x'!

3.  x*! e y'!

4.  (A@x : (@x e x'! -> @x e y'!))
----



0.  x*! e z'!

by lines [-1]
Next!"""

left()

"""

Line number 15



0.  (A@x : (@x e y'! -> @x e z'!))

1.  x*! e x'!

2.  x*! e y'!

3.  (A@x : (@x e x'! -> @x e y'!))
----
```

```
0.  x'*? e y'!

1.  x*! e z'!

by lines [-1]
Next!"""

setunknown ("x'*","*x")

"""

Line number 15



0.  (A@x : (@x e y'! -> @x e z'!))

1.  x*! e x'!

2.  x*! e y'!

3.  (A@x : (@x e x'! -> @x e y'!))
----



0.  x*! e y'!

1.  x*! e z'!

by lines [-1]
Next!"""

getleft(2)

"""

Line number 15
```

```
0.  x*! e y'!

1.  (A@x : (@x e x'! -> @x e y'!))

2.  (A@x : (@x e y'! -> @x e z'!))

3.  x*! e x'!
----



0.  x*! e y'!

1.  x*! e z'!

by lines [-1]
Next!"""

done()

"""

Line number 16




0.  x*! e z'!

1.  (A@x : (@x e y'! -> @x e z'!))

2.  x*! e x'!

3.  x*! e y'!

4.  (A@x : (@x e x'! -> @x e y'!))
----
```

```
0.  x*! e z'!

by lines [-1]
Next!"""

done()

"""Done!"""

"""

Line number 0



----



0.  (Ax : (Ay : (Az : (((x C y) & (y C z)) -> (x C z)))))

by lines [1]

Line number 1



----



0.  (Ay : (Az : (((x'! C y) & (y C z)) -> (x'! C z))))

by lines [2]

Line number 2
```

```
----


0.  (Az : (((x'! C y'!) & (y'! C z)) -> (x'! C z)))

by lines [3]

Line number 3


----


0.  (((x'! C y'!) & (y'! C z'!)) -> (x'! C z'!))

by lines [4]

Line number 4


0.  ((x'! C y'!) & (y'! C z'!))
----


0.  (x'! C z'!)

by lines [5]

Line number 5
```

```
0.  ((x'! C y'!) & (y'! C z'!))
----



0.  (A@x : (@x e x'! -> @x e z'!))

by lines [6]

Line number 6



0.  ((x'! C y'!) & (y'! C z'!))
----



0.  (x*! e x'! -> x*! e z'!)

by lines [7]

Line number 7



0.  ((x'! C y'!) & (y'! C z'!))

1.  x*! e x'!
----



0.  x*! e z'!

by lines [8]
```

```
Line number 8



0.  (x'! C y'!)

1.  (y'! C z'!)

2.  x*! e x'!
----



0.  x*! e z'!

by lines [9]

Line number 9



0.  (A@x : (@x e x'! -> @x e y'!))

1.  (y'! C z'!)

2.  x*! e x'!
----



0.  x*! e z'!

by lines [10]

Line number 10
```

```
0.  (x*! e x'! -> x*! e y'!)

1.  (A@x : (@x e x'! -> @x e y'!))

2.  (y'! C z'!)

3.  x*! e x'!
----



0.  x*! e z'!

by lines [11, 12]

Line number 11



0.  x*! e x'!

1.  (A@x : (@x e x'! -> @x e y'!))

2.  (y'! C z'!)
----



0.  x*! e x'!

1.  x*! e z'!

by lines []

Line number 12
```

```
0.  (y'! C z'!)

1.  x*! e x'!

2.  x*! e y'!

3.  (A@x : (@x e x'! -> @x e y'!))
----



0.  x*! e z'!

by lines [13]

Line number 13



0.  (A@x : (@x e y'! -> @x e z'!))

1.  x*! e x'!

2.  x*! e y'!

3.  (A@x : (@x e x'! -> @x e y'!))
----



0.  x*! e z'!

by lines [14]

Line number 14
```

```
0.  (x*! e y'! -> x*! e z'!)

1.  (A@x : (@x e y'! -> @x e z'!))

2.  x*! e x'!

3.  x*! e y'!

4.  (A@x : (@x e x'! -> @x e y'!))
----



0.  x*! e z'!

by lines [15, 16]

Line number 15



0.  x*! e y'!

1.  (A@x : (@x e x'! -> @x e y'!))

2.  (A@x : (@x e y'! -> @x e z'!))

3.  x*! e x'!
----



0.  x*! e y'!

1.  x*! e z'!

by lines []
```

```
Line number 16



0.  x*! e z'!

1.  (A@x : (@x e y'! -> @x e z'!))

2.  x*! e x'!

3.  x*! e y'!

4.  (A@x : (@x e x'! -> @x e y'!))
----



0.  x*! e z'!

by lines []"""
```

7.2 Equality is transitive

Here is a proof that equality is transitive.

```
from graph import *
setlog("eqtranitive")
start ("AxAyAz>&=xy=yz=xz")

"""

Line number 0



----
```

```
0.  (Ax : (Ay : (Az : ((x = y & y = z) -> x = z))))

by lines [-1]"""
right()

"""

Line number 1



----



0.  (Ay : (Az : ((x'! = y & y = z) -> x'! = z)))

by lines [-1]"""
right()

"""

Line number 2



----



0.  (Az : ((x'! = y'! & y'! = z) -> x'! = z))

by lines [-1]"""
right()

"""

Line number 3
```

```
----



0.  ((x'! = y'! & y'! = z'!) -> x'! = z'!)

by lines [-1]"""
right()

"""

Line number 4




0.  (x'! = y'! & y'! = z'!)
----



0.  x'! = z'!

by lines [-1]"""
right()

"""

Line number 5




0.  (x'! = y'! & y'! = z'!)
----



0.  (Ax* : (x* e x'! == x* e z'!))
```

```
by lines [-1]"""
right()

"""

Line number 6



0.  (x'! = y'! & y'! = z'!)
----



0.  (x''! e x'! == x''! e z'!)

by lines [-1]"""
right()

"""

Line number 7



0.  x''! e x'!

1.  (x'! = y'! & y'! = z'!)
----



0.  x''! e z'!

by lines [-1]"""
getleft(1)
```

```
"""

Line number 7



0.  (x'! = y'! & y'! = z'!)

1.  x''! e x'!
----



0.  x''! e z'!

by lines [-1]"""
left()

"""

Line number 9



0.  x'! = y'!

1.  y'! = z'!

2.  x''! e x'!
----



0.  x''! e z'!

by lines [-1]"""
left()
```

```
"""

Line number 10



0.  (Ax'* : (x'! e x'* == y'! e x'*))

1.  y'! = z'!

2.  x''! e x'!
----



0.  x''! e z'!

by lines [-1]"""
left()

"""

Line number 11



0.  (x'! e x*'? == y'! e x*'?)

1.  (Ax'* : (x'! e x'* == y'! e x'*))

2.  y'! = z'!

3.  x''! e x'!
----



0.  x''! e z'!
```

```
by lines [-1]"""
left()

"""

Line number 12



0.  (x'! e x*'? -> y'! e x*'?)

1.  (y'! e x*'? -> x'! e x*'?)

2.  (Ax'* : (x'! e x'* == y'! e x'*))

3.  y'! = z'!

4.  x''! e x'!
----



0.  x''! e z'!

by lines [-1]"""
left()

"""

Line number 13



0.  (y'! e x*'? -> x'! e x*'?)

1.  (Ax'* : (x'! e x'* == y'! e x'*))
```

```
2.  y'! = z'!

3.  x''! e x'!
----



0.  x'! e x*'?

1.  x''! e z'!

by lines [-1]"""
setunknown ("x*'","{ue''xu")

"""

Line number 13



0.  (y'! e {u | x''! e u} -> x'! e {u | x''! e u})

1.  (Ax'* : (x'! e x'* == y'! e x'*))

2.  y'! = z'!

3.  x''! e x'!
----



0.  x'! e {u | x''! e u}

1.  x''! e z'!

by lines [-1]"""
right()

"""
```

```
Line number 15



0.  (y'! e {u | x''! e u} -> x'! e {u | x''! e u})

1.  (Ax'* : (x'! e x'* == y'! e x'*))

2.  y'! = z'!

3.  x''! e x'!
----



0.  x''! e x'!

1.  x''! e z'!

by lines [-1]"""
getleft(3)

"""

Line number 15



0.  x''! e x'!

1.  (y'! e {u | x''! e u} -> x'! e {u | x''! e u})

2.  (Ax'* : (x'! e x'* == y'! e x'*))

3.  y'! = z'!
----
```

```
0.  x''! e x'!

1.  x''! e z'!

by lines [-1]"""
done()

"""

Line number 14



0.  y'! e {u | x''! e u}

1.  (y'! e {u | x''! e u} -> x'! e {u | x''! e u})

2.  (Ax'* : (x'! e x'* == y'! e x'*))

3.  y'! = z'!

4.  x''! e x'!
----



0.  x''! e z'!

by lines [-1]"""
left()

"""

Line number 16
```

```
0.  x''! e y'!

1.  (y'! e {u | x''! e u} -> x'! e {u | x''! e u})

2.  (Ax'* : (x'! e x'* == y'! e x'*))

3.  y'! = z'!

4.  x''! e x'!
----



0.  x''! e z'!

by lines [-1]"""
getleft(3)

"""

Line number 16



0.  y'! = z'!

1.  x''! e x'!

2.  x''! e y'!

3.  (y'! e {u | x''! e u} -> x'! e {u | x''! e u})

4.  (Ax'* : (x'! e x'* == y'! e x'*))
----



0.  x''! e z'!
```

```
by lines [-1]"""
left()

"""

Line number 17



0.  (Ax** : (y'! e x** == z'! e x**))

1.  x''! e x'!

2.  x''! e y'!

3.  (y'! e {u | x''! e u} -> x'! e {u | x''! e u})

4.  (Ax'* : (x'! e x'* == y'! e x'*))
----



0.  x''! e z'!

by lines [-1]"""
left()

"""

Line number 18



0.  (y'! e x'''? == z'! e x'''?)

1.  (Ax** : (y'! e x** == z'! e x**))
```

```
2.  x''! e x'!

3.  x''! e y'!

4.  (y'! e {u | x''! e u} -> x'! e {u | x''! e u})

5.  (Ax'* : (x'! e x'* == y'! e x'*))
----



0.  x''! e z'!

by lines [-1]"""
left()

"""

Line number 19



0.  (y'! e x'''? -> z'! e x'''?)

1.  (z'! e x'''? -> y'! e x'''?)

2.  (Ax** : (y'! e x** == z'! e x**))

3.  x''! e x'!

4.  x''! e y'!

5.  (y'! e {u | x''! e u} -> x'! e {u | x''! e u})

6.  (Ax'* : (x'! e x'* == y'! e x'*))
----
```

```
0.  x''! e z'!

by lines [-1]"""
left()

"""

Line number 20



0.  (z'! e x'''? -> y'! e x'''?)

1.  (Ax** : (y'! e x** == z'! e x**))

2.  x''! e x'!

3.  x''! e y'!

4.  (y'! e {u | x''! e u} -> x'! e {u | x''! e u})

5.  (Ax'* : (x'! e x'* == y'! e x'*))
----



0.  y'! e x'''?

1.  x''! e z'!

by lines [-1]"""
setunknown ("x''''","{ue''xu")

"""

Line number 20
```

```
0.  (z'! e {u | x''! e u} -> y'! e {u | x''! e u})

1.  (Ax** : (y'! e x** == z'! e x**))

2.  x''! e x'!

3.  x''! e y'!

4.  (y'! e {u | x''! e u} -> x'! e {u | x''! e u})

5.  (Ax'* : (x'! e x'* == y'! e x'*))
----



0.  y'! e {u | x''! e u}

1.  x''! e z'!

by lines [-1]"""
right()

"""

Line number 22



0.  (z'! e {u | x''! e u} -> y'! e {u | x''! e u})

1.  (Ax** : (y'! e x** == z'! e x**))

2.  x''! e x'!

3.  x''! e y'!

4.  (y'! e {u | x''! e u} -> x'! e {u | x''! e u})
```

```
5.  (Ax'* : (x'! e x'* == y'! e x'*))
----



0.  x''! e y'!

1.  x''! e z'!

by lines [-1]"""
getleft(3)

"""

Line number 22



0.  x''! e y'!

1.  (y'! e {u | x''! e u} -> x'! e {u | x''! e u})

2.  (Ax'* : (x'! e x'* == y'! e x'*))

3.  (z'! e {u | x''! e u} -> y'! e {u | x''! e u})

4.  (Ax** : (y'! e x** == z'! e x**))

5.  x''! e x'!
----



0.  x''! e y'!

1.  x''! e z'!

by lines [-1]"""
done()
```

```
"""

Line number 21



0.  z'! e {u | x''! e u}

1.  (z'! e {u | x''! e u} -> y'! e {u | x''! e u})

2.  (Ax** : (y'! e x** == z'! e x**))

3.  x''! e x'!

4.  x''! e y'!

5.  (y'! e {u | x''! e u} -> x'! e {u | x''! e u})

6.  (Ax'* : (x'! e x'* == y'! e x'*))
----



0.  x''! e z'!

by lines [-1]"""
left()

"""

Line number 23



0.  x''! e z'!

1.  (z'! e {u | x''! e u} -> y'! e {u | x''! e u})
```

```
2.  (Ax** : (y'! e x** == z'! e x**))

3.  x''! e x'!

4.  x''! e y'!

5.  (y'! e {u | x''! e u} -> x'! e {u | x''! e u})

6.  (Ax'* : (x'! e x'* == y'! e x'*))
----



0.  x''! e z'!

by lines [-1]"""
done()

"""

Line number 8



0.  x''! e z'!

1.  (x'! = y'! & y'! = z'!)
----



0.  x''! e x'!

by lines [-1]"""
getleft(1)

"""
```

```
Line number 8



0.  (x'! = y'! & y'! = z'!)

1.  x''! e z'!
----



0.  x''! e x'!

by lines [-1]"""
left()

"""

Line number 24



0.  x'! = y'!

1.  y'! = z'!

2.  x''! e z'!
----



0.  x''! e x'!

by lines [-1]"""
getleft(1)

"""
```

```
Line number 24



0.  y'! = z'!

1.  x''! e z'!

2.  x'! = y'!
----



0.  x''! e x'!

by lines [-1]"""
left()

"""

Line number 25



0.  (Ax''* : (y'! e x''* == z'! e x''*))

1.  x''! e z'!

2.  x'! = y'!
----



0.  x''! e x'!

by lines [-1]"""
left()
```

```
"""

Line number 26



0.  (y'! e x'*'? == z'! e x'*'?)

1.  (Ax''* : (y'! e x''* == z'! e x''*))

2.  x''! e z'!

3.  x'! = y'!
----



0.  x''! e x'!

by lines [-1]"""
left()

"""

Line number 27



0.  (y'! e x'*'? -> z'! e x'*'?)

1.  (z'! e x'*'? -> y'! e x'*'?)

2.  (Ax''* : (y'! e x''* == z'! e x''*))

3.  x''! e z'!

4.  x'! = y'!
----
```

```
0.  x''! e x'!

by lines [-1]"""
getleft(1)

"""

Line number 27



0.  (z'! e x'*'? -> y'! e x'*'?)

1.  (Ax''* : (y'! e x''* == z'! e x''*))

2.  x''! e z'!

3.  x'! = y'!

4.  (y'! e x'*'? -> z'! e x'*'?)
----



0.  x''! e x'!

by lines [-1]"""
left()

"""

Line number 28
```

```
0.  (Ax''* : (y'! e x''* == z'! e x''*))

1.  x''! e z'!

2.  x'! = y'!

3.  (y'! e x'*'? -> z'! e x'*'?)
----



0.  z'! e x'*'?

1.  x''! e x'!

by lines [-1]"""
setunknown ("x'*'","{ue''xu")

"""

Line number 28



0.  (Ax''* : (y'! e x''* == z'! e x''*))

1.  x''! e z'!

2.  x'! = y'!

3.  (y'! e {u | x''! e u} -> z'! e {u | x''! e u})
----



0.  z'! e {u | x''! e u}

1.  x''! e x'!
```

```
by lines [-1]"""
right()

"""

Line number 30



0.  (Ax''* : (y'! e x''* == z'! e x''*))

1.  x''! e z'!

2.  x'! = y'!

3.  (y'! e {u | x''! e u} -> z'! e {u | x''! e u})
----



0.  x''! e z'!

1.  x''! e x'!

by lines [-1]"""
getleft(1)

"""

Line number 30



0.  x''! e z'!

1.  x'! = y'!

2.  (y'! e {u | x''! e u} -> z'! e {u | x''! e u})
```

```
3.  (Ax''* : (y'! e x''* == z'! e x''*))
----



0.  x''! e z'!

1.  x''! e x'!

by lines [-1]"""
done()

"""

Line number 29



0.  y'! e {u | x''! e u}

1.  (Ax''* : (y'! e x''* == z'! e x''*))

2.  x''! e z'!

3.  x'! = y'!

4.  (y'! e {u | x''! e u} -> z'! e {u | x''! e u})
----



0.  x''! e x'!

by lines [-1]"""
left()

"""
```

```
Line number 31



0.  x''! e y'!

1.  (Ax''* : (y'! e x''* == z'! e x''*))

2.  x''! e z'!

3.  x'! = y'!

4.  (y'! e {u | x''! e u} -> z'! e {u | x''! e u})
----



0.  x''! e x'!

by lines [-1]"""
getleft(3)

"""

Line number 31



0.  x'! = y'!

1.  (y'! e {u | x''! e u} -> z'! e {u | x''! e u})

2.  x''! e y'!

3.  (Ax''* : (y'! e x''* == z'! e x''*))

4.  x''! e z'!
----
```

```
0.  x''! e x'!

by lines [-1]"""
left()

"""

Line number 32



0.  (Ax'** : (x'! e x'** == y'! e x'**))

1.  (y'! e {u | x''! e u} -> z'! e {u | x''! e u})

2.  x''! e y'!

3.  (Ax''* : (y'! e x''* == z'! e x''*))

4.  x''! e z'!
----



0.  x''! e x'!

by lines [-1]"""
left()

"""

Line number 33
```

```
0.  (x'! e x*''? == y'! e x*''?)

1.  (Ax'** : (x'! e x'** == y'! e x'**))

2.  (y'! e {u | x''! e u} -> z'! e {u | x''! e u})

3.  x''! e y'!

4.  (Ax''* : (y'! e x''* == z'! e x''*))

5.  x''! e z'!
----



0.  x''! e x'!

by lines [-1]"""
left()

"""

Line number 34



0.  (x'! e x*''? -> y'! e x*''?)

1.  (y'! e x*''? -> x'! e x*''?)

2.  (Ax'** : (x'! e x'** == y'! e x'**))

3.  (y'! e {u | x''! e u} -> z'! e {u | x''! e u})

4.  x''! e y'!

5.  (Ax''* : (y'! e x''* == z'! e x''*))

6.  x''! e z'!
```

```
----



0.  x''! e x'!

by lines [-1]"""
getleft(1)

"""

Line number 34



0.  (y'! e x*''? -> x'! e x*''?)

1.  (Ax'** : (x'! e x'** == y'! e x'**))

2.  (y'! e {u | x''! e u} -> z'! e {u | x''! e u})

3.  x''! e y'!

4.  (Ax''* : (y'! e x''* == z'! e x''*))

5.  x''! e z'!

6.  (x'! e x*''? -> y'! e x*''?)
----



0.  x''! e x'!

by lines [-1]"""
left()

"""
```

```
Line number 35



0.  (Ax'** : (x'! e x'** == y'! e x'**))

1.  (y'! e {u | x''! e u} -> z'! e {u | x''! e u})

2.  x''! e y'!

3.  (Ax''* : (y'! e x''* == z'! e x''*))

4.  x''! e z'!

5.  (x'! e x*''? -> y'! e x*''?)
----



0.  y'! e x*''?

1.  x''! e x'!

by lines [-1]"""
setunknown ("x*''","{ue''xu")

"""

Line number 35



0.  (Ax'** : (x'! e x'** == y'! e x'**))

1.  (y'! e {u | x''! e u} -> z'! e {u | x''! e u})

2.  x''! e y'!
```

```
3.  (Ax''* : (y'! e x''* == z'! e x''*))

4.  x''! e z'!

5.  (x'! e {u | x''! e u} -> y'! e {u | x''! e u})
----



0.  y'! e {u | x''! e u}

1.  x''! e x'!

by lines [-1]"""
right()

"""

Line number 37



0.  (Ax'** : (x'! e x'** == y'! e x'**))

1.  (y'! e {u | x''! e u} -> z'! e {u | x''! e u})

2.  x''! e y'!

3.  (Ax''* : (y'! e x''* == z'! e x''*))

4.  x''! e z'!

5.  (x'! e {u | x''! e u} -> y'! e {u | x''! e u})
----



0.  x''! e y'!
```

```
1.  x''! e x'!

by lines [-1]"""
getleft(2)

"""

Line number 37



0.  x''! e y'!

1.  (Ax''* : (y'! e x''* == z'! e x''*))

2.  x''! e z'!

3.  (x'! e {u | x''! e u} -> y'! e {u | x''! e u})

4.  (Ax'** : (x'! e x'** == y'! e x'**))

5.  (y'! e {u | x''! e u} -> z'! e {u | x''! e u})
----



0.  x''! e y'!

1.  x''! e x'!

by lines [-1]"""
done()

"""

Line number 36
```

```
0.  x'! e {u | x''! e u}

1.  (Ax'** : (x'! e x'** == y'! e x'**))

2.  (y'! e {u | x''! e u} -> z'! e {u | x''! e u})

3.  x''! e y'!

4.  (Ax''* : (y'! e x''* == z'! e x''*))

5.  x''! e z'!

6.  (x'! e {u | x''! e u} -> y'! e {u | x''! e u})
----



0.  x''! e x'!

by lines [-1]"""
left()

"""

Line number 38



0.  x''! e x'!

1.  (Ax'** : (x'! e x'** == y'! e x'**))

2.  (y'! e {u | x''! e u} -> z'! e {u | x''! e u})

3.  x''! e y'!

4.  (Ax''* : (y'! e x''* == z'! e x''*))
```

```
5.  x''! e z'!

6.  (x'! e {u | x''! e u} -> y'! e {u | x''! e u})
----



0.  x''! e x'!

by lines [-1]"""
done()
setlog("done")
```

7.3 Coextensionality implies Leibniz equality

This example is perhaps a bit strange. This is a proof that coextensionality implies Leibniz equality: it uses the weird left rule for membership statements which I have introduced experimentally, in order to avoid the need to cut. It is also a nice sequent proof, and examplifies the notation and style of use of the prover, so here it is.

This new version uses the new higher order matching facility.

```
from graph import *

start ("AaAb>AxXexaexbAuXeauebu")

"""

Line number 0



----



0.  (Aa : (Ab : ((Ax : (x e a == x e b)) -> (Au : (a e u == b e u)))))

by lines [-1]
Next!"""
```

```
right()

"""

Line number 1



----



0.  (Ab : ((Ax : (x e a'! == x e b)) -> (Au : (a'! e u == b e u))))

by lines [-1]
Next!"""

right()

"""

Line number 2



----



0.  ((Ax : (x e a'! == x e b'!)) -> (Au : (a'! e u == b'! e u)))

by lines [-1]
Next!"""

right()

"""

Line number 3
```

```
0.  (Ax : (x e a'! == x e b'!))
----



0.  (Au : (a'! e u == b'! e u))

by lines [-1]
Next!"""

right()

"""

Line number 4



0.  (Ax : (x e a'! == x e b'!))
----



0.  (a'! e u'! == b'! e u'!)

by lines [-1]
Next!"""

right()

"""

Line number 5
```

```
0.  a'! e u'!

1.  (Ax : (x e a'! == x e b'!))
----



0.  b'! e u'!

by lines [-1]
Next!"""

left()

"""

Line number 7



0.  (Ax' : ((Ax* : (x* e a'! == x* e x')) -> x' e u'!))

1.  (Ax : (x e a'! == x e b'!))
----



0.  b'! e u'!

by lines [-1]
Next!"""

left()

"""

Line number 8
```

```
0.  ((Ax* : (x* e a'! == x* e x''?)) -> x''? e u'!)

1.  (Ax' : ((Ax* : (x* e a'! == x* e x')) -> x' e u'!))

2.  (Ax : (x e a'! == x e b'!))
----



0.  b'! e u'!

by lines [-1]
Next!"""

left()

"""

Line number 9



0.  (Ax' : ((Ax* : (x* e a'! == x* e x')) -> x' e u'!))

1.  (Ax : (x e a'! == x e b'!))
----



0.  (Ax* : (x* e a'! == x* e x''?))

1.  b'! e u'!

by lines [-1]
Next!"""
```

```
left()

"""

Line number 11



0.  ((Ax* : (x* e a'! == x* e x'*?)) -> x'*? e u'!)

1.  (Ax' : ((Ax* : (x* e a'! == x* e x')) -> x' e u'!))

2.  (Ax : (x e a'! == x e b'!))
----



0.  (Ax* : (x* e a'! == x* e x''?))

1.  b'! e u'!

by lines [-1]
Next!"""

getleft(2)

"""

Line number 11



0.  (Ax : (x e a'! == x e b'!))

1.  ((Ax* : (x* e a'! == x* e x'*?)) -> x'*? e u'!)

2.  (Ax' : ((Ax* : (x* e a'! == x* e x')) -> x' e u'!))
```

```
----



0.  (Ax* : (x* e a'! == x* e x''?))

1.  b'! e u'!

by lines [-1]
Next!"""

done()

"""

Line number 10




0.  b'! e u'!

1.  (Ax' : ((Ax* : (x* e a'! == x* e x')) -> x' e u'!))

2.  (Ax : (x e a'! == x e b'!))
----



0.  b'! e u'!

by lines [-1]
Next!"""

done()

"""

Line number 6
```

```
0.  b'! e u'!

1.  (Ax : (x e a'! == x e b'!))
----



0.  a'! e u'!

by lines [-1]
Next!"""

left()

"""

Line number 12



0.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))

1.  (Ax : (x e a'! == x e b'!))
----



0.  a'! e u'!

by lines [-1]
Next!"""

left()

"""
```

```
Line number 13



0.  ((Ax''' : (x''' e b'! == x''' e x''*?)) -> x''*? e u'!)

1.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))

2.  (Ax : (x e a'! == x e b'!))
----



0.  a'! e u'!

by lines [-1]
Next!"""

left()

"""

Line number 14



0.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))

1.  (Ax : (x e a'! == x e b'!))
----



0.  (Ax''' : (x''' e b'! == x''' e x''*?))

1.  a'! e u'!

by lines [-1]
```

```
Next!"""

getleft(1)

"""

Line number 14



0.  (Ax : (x e a'! == x e b'!))

1.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))
----



0.  (Ax''' : (x''' e b'! == x''' e x''*?))

1.  a'! e u'!

by lines [-1]
Next!"""

right()

"""

Line number 16



0.  (Ax : (x e a'! == x e b'!))

1.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))
----
```

```
0.  (x'**! e b'! == x'**! e x''*?)

1.  a'! e u'!

by lines [-1]
Next!"""

right()

"""

Line number 17




0.  x'**! e b'!

1.  (Ax : (x e a'! == x e b'!))

2.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))
----



0.  x'**! e x''*?

1.  a'! e u'!

by lines [-1]
Next!"""

done()

"""

Line number 18
```

```
0.  x'**! e b'!

1.  (Ax : (x e a'! == x e b'!))

2.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))
----



0.  x'**! e b'!

1.  a'! e u'!

by lines [-1]
Next!"""

done()

"""

Line number 15




0.  b'! e u'!

1.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))

2.  (Ax : (x e a'! == x e b'!))
----



0.  a'! e u'!

by lines [-1]
Next!"""
```

```
getleft(2)

"""

Line number 15



0.  (Ax : (x e a'! == x e b'!))

1.  b'! e u'!

2.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))
----



0.  a'! e u'!

by lines [-1]
Next!"""

left()

"""

Line number 19



0.  (x*''? e a'! == x*''? e b'!)

1.  (Ax : (x e a'! == x e b'!))

2.  b'! e u'!

3.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))
```

```
----



0.  a'! e u'!

by lines [-1]
Next!"""

left()

"""

Line number 20




0.  (x*''? e a'! -> x*''? e b'!)

1.  (x*''? e b'! -> x*''? e a'!)

2.  (Ax : (x e a'! == x e b'!))

3.  b'! e u'!

4.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))
----



0.  a'! e u'!

by lines [-1]
Next!"""

getleft(1)

"""
```

```
Line number 20



0.  (x*''? e b'! -> x*''? e a'!)

1.  (Ax : (x e a'! == x e b'!))

2.  b'! e u'!

3.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))

4.  (x*''? e a'! -> x*''? e b'!)
----



0.  a'! e u'!

by lines [-1]
Next!"""

left()

"""

Line number 21



0.  (Ax : (x e a'! == x e b'!))

1.  b'! e u'!

2.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))

3.  (x*''? e a'! -> x*''? e b'!)
----
```

```
0.  x*''? e b'!

1.  a'! e u'!

by lines [-1]
Next!"""

getleft(1)

"""

Line number 21



0.  b'! e u'!

1.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))

2.  (x*''? e a'! -> x*''? e b'!)

3.  (Ax : (x e a'! == x e b'!))
----



0.  x*''? e b'!

1.  a'! e u'!

by lines [-1]
Next!"""

back()
back()
back()
```

```
back()
back()
back()
getleft(2)

"""

Line number 15



0.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))

1.  (Ax : (x e a'! == x e b'!))

2.  b'! e u'!
----



0.  a'! e u'!

by lines [-1]
Next!"""

left()

"""

Line number 19



0.  ((Ax''' : (x''' e b'! == x''' e x*''?)) -> x*''? e u'!)

1.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))

2.  (Ax : (x e a'! == x e b'!))
```

```
3.  b’! e u’!
----



0.  a’! e u’!

by lines [-1]
Next!"""

left()

"""

Line number 20



0.  (Ax** : ((Ax’’’ : (x’’’ e b’! == x’’’ e x**)) -> x** e u’!))

1.  (Ax : (x e a’! == x e b’!))

2.  b’! e u’!
----



0.  (Ax’’’ : (x’’’ e b’! == x’’’ e x*’’?))

1.  a’! e u’!

by lines [-1]
Next!"""

right()

"""
```

```
Line number 22



0.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))

1.  (Ax : (x e a'! == x e b'!))

2.  b'! e u'!
----



0.  (x*'*! e b'! == x*'*! e x*''?)

1.  a'! e u'!

by lines [-1]
Next!"""

right()

"""

Line number 23



0.  x*'*! e b'!

1.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))

2.  (Ax : (x e a'! == x e b'!))

3.  b'! e u'!
----
```

```
0.  x*'*! e x*''?

1.  a'! e u'!

by lines [-1]
Next!"""

setunknown ("x*''","'a")

"""

Line number 23



0.  x*'*! e b'!

1.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))

2.  (Ax : (x e a'! == x e b'!))

3.  b'! e u'!
----



0.  x*'*! e a'!

1.  a'! e u'!

by lines [-1]
Next!"""

getleft(2)

"""

Line number 23
```

```
0.  (Ax : (x e a'! == x e b'!))

1.  b'! e u'!

2.  x*'*! e b'!

3.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))
----



0.  x*'*! e a'!

1.  a'! e u'!

by lines [-1]
Next!"""

left()

"""

Line number 25




0.  (x**'? e a'! == x**'? e b'!)

1.  (Ax : (x e a'! == x e b'!))

2.  b'! e u'!

3.  x*'*! e b'!

4.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))
```

```
----


0.  x*'*! e a'!

1.  a'! e u'!

by lines [-1]
Next!"""

left()

"""

Line number 26



0.  (x**'? e a'! -> x**'? e b'!)

1.  (x**'? e b'! -> x**'? e a'!)

2.  (Ax : (x e a'! == x e b'!))

3.  b'! e u'!

4.  x*'*! e b'!

5.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))
----


0.  x*'*! e a'!

1.  a'! e u'!

by lines [-1]
```

```
Next!"""

getleft(1)

"""

Line number 26



0.  (x**'? e b'! -> x**'? e a'!)

1.  (Ax : (x e a'! == x e b'!))

2.  b'! e u'!

3.  x*'*! e b'!

4.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))

5.  (x**'? e a'! -> x**'? e b'!)
----



0.  x*'*! e a'!

1.  a'! e u'!

by lines [-1]
Next!"""

left()

"""

Line number 27
```

```
0.  (Ax : (x e a'! == x e b'!))

1.  b'! e u'!

2.  x*'*! e b'!

3.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))

4.  (x**'? e a'! -> x**'? e b'!)
----



0.  x**'? e b'!

1.  x*'*! e a'!

2.  a'! e u'!

by lines [-1]
Next!"""

getleft(2)

"""

Line number 27



0.  x*'*! e b'!

1.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))

2.  (x**'? e a'! -> x**'? e b'!)

3.  (Ax : (x e a'! == x e b'!))
```

```
4.  b'! e u'!
----



0.  x**'? e b'!

1.  x*'*! e a'!

2.  a'! e u'!

by lines [-1]
Next!"""

done()

"""

Line number 28



0.  x*'*! e a'!

1.  (Ax : (x e a'! == x e b'!))

2.  b'! e u'!

3.  x*'*! e b'!

4.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))

5.  (x*'*! e a'! -> x*'*! e b'!)
----



0.  x*'*! e a'!
```

```
1.  a'! e u'!

by lines [-1]
Next!"""

done()

"""

Line number 24



0.  x*'*! e a'!

1.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))

2.  (Ax : (x e a'! == x e b'!))

3.  b'! e u'!
----



0.  x*'*! e b'!

1.  a'! e u'!

by lines [-1]
Next!"""

getleft(2)

"""

Line number 24
```

```
0.  (Ax : (x e a'! == x e b'!))

1.  b'! e u'!

2.  x*'*! e a'!

3.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))
----



0.  x*'*! e b'!

1.  a'! e u'!

by lines [-1]
Next!"""

left()

"""

Line number 29



0.  (x***? e a'! == x***? e b'!)

1.  (Ax : (x e a'! == x e b'!))

2.  b'! e u'!

3.  x*'*! e a'!

4.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))
----
```

```
0.  x*'*! e b'!

1.  a'! e u'!

by lines [-1]
Next!"""

left()

"""

Line number 30



0.  (x***? e a'! -> x***? e b'!)

1.  (x***? e b'! -> x***? e a'!)

2.  (Ax : (x e a'! == x e b'!))

3.  b'! e u'!

4.  x*'*! e a'!

5.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))
----



0.  x*'*! e b'!

1.  a'! e u'!

by lines [-1]
Next!"""
```

```
left()

"""

Line number 31



0.  (x***? e b'! -> x***? e a'!)

1.  (Ax : (x e a'! == x e b'!))

2.  b'! e u'!

3.  x*'*! e a'!

4.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))
----



0.  x***? e a'!

1.  x*'*! e b'!

2.  a'! e u'!

by lines [-1]
Next!"""

getleft(3)

"""

Line number 31
```

```
0.  x*'*! e a'!

1.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))

2.  (x***? e b'! -> x***? e a'!)

3.  (Ax : (x e a'! == x e b'!))

4.  b'! e u'!
----



0.  x***? e a'!

1.  x*'*! e b'!

2.  a'! e u'!

by lines [-1]
Next!"""

done()

"""

Line number 32



0.  x*'*! e b'!

1.  (x*'*! e b'! -> x*'*! e a'!)

2.  (Ax : (x e a'! == x e b'!))

3.  b'! e u'!

4.  x*'*! e a'!
```

```
5.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))
----



0.  x*'*! e b'!

1.  a'! e u'!

by lines [-1]
Next!"""

done()

"""

Line number 21



0.  a'! e u'!

1.  (Ax** : ((Ax''' : (x''' e b'! == x''' e x**)) -> x** e u'!))

2.  (Ax : (x e a'! == x e b'!))

3.  b'! e u'!
----



0.  a'! e u'!

by lines [-1]
Next!"""

done()
```

```
"""Done!"""
```

7.4 A weird example with unknown variables

The entry line illustrates the new capability to use spaces, parentheses, brackets, and commas as padding in term and formula input. There is no balance checking: parentheses and brackets are just for the person entering the term [to make sure they do it right].

```
from graph import *

start ("AyEuAx>exyeuy")

"""

Line number 0



----



0.  (Ay : (Eu : (Ax : (x e y -> u e y))))

by lines [-1]
Next!"""

right()

"""

Line number 1



----
```

```
0.  (Eu : (Ax : (x e y'! -> u e y'!)))

by lines [-1]
Next!"""

right()

"""

Line number 2



----



0.  (Ax : (x e y' -> u'? e y'))

1.  (Eu : (Ax : (x e y' -> u e y')))

by lines [-1]
Next!"""

right()

"""

Line number 3



----



0.  (x'! e y' -> u'? e y')

1.  (Eu : (Ax : (x e y' -> u e y')))
```

```
by lines [-1]
Next!"""

right()

"""

Line number 4




0.  x'! e y'
----



0.  u'? e y'

1.  (Eu : (Ax : (x e y' -> u e y')))

by lines [-1]
Next!"""

done()
getright(1)

"""

Line number 4




0.  x'! e y'
----
```

```
0.  (Eu : (Ax : (x e y' -> u e y')))

1.  u'? e y'

by lines [-1]
Next!"""

right()

"""

Line number 5




0.  x' e y'
----



0.  (Ax : (x e y' -> u*? e y'))

1.  (Eu : (Ax : (x e y' -> u e y')))

2.  u'? e y'

by lines [-1]
Next!"""

right()

"""

Line number 6




0.  x' e y'
```

```
----



0.  (x*! e y' -> u*? e y')

1.  (Eu : (Ax : (x e y' -> u e y')))

2.  u'? e y'

by lines [-1]
Next!"""

right()

"""

Line number 7




0.  x*! e y'

1.  x' e y'
----



0.  u*? e y'

1.  (Eu : (Ax : (x e y' -> u e y')))

2.  u'? e y'

by lines [-1]
Next!"""

getleft(1)
```

```
"""

Line number 7



0.  x' e y'

1.  x*! e y'
----



0.  u*? e y'

1.  (Eu : (Ax : (x e y' -> u e y')))

2.  u'? e y'

by lines [-1]
Next!"""

done()

"""Done!"""

"""

Line number 0



----



0.  (Ay : (Eu : (Ax : (x e y -> u e y))))

by lines [1]
```

```
Line number 1


----


0.  (Eu : (Ax : (x e y' -> u e y')))

by lines [2]

Line number 2


----


0.  (Ax : (x e y' -> u'? e y'))

1.  (Eu : (Ax : (x e y' -> u e y')))

by lines [3]

Line number 3


----


0.  (x' e y' -> u'? e y')

1.  (Eu : (Ax : (x e y' -> u e y')))

by lines [4]
```

```
Line number 4



0.  x' e y'
----



0.  (Eu : (Ax : (x e y' -> u e y')))

1.  u'? e y'

by lines [5]

Line number 5



0.  x' e y'
----



0.  (Ax : (x e y' -> x' e y'))

1.  (Eu : (Ax : (x e y' -> u e y')))

2.  u'? e y'

by lines [6]

Line number 6
```

```
0.  x' e y'
----



0.  (x*! e y' -> x' e y')

1.  (Eu : (Ax : (x e y' -> u e y')))

2.  u'? e y'

by lines [7]

Line number 7



0.  x' e y'

1.  x*! e y'
----



0.  x' e y'

1.  (Eu : (Ax : (x e y' -> u e y')))

2.  u'? e y'

by lines []"""
```

7.5 Atomic inclusion is membership

This proves the equivalence of $\{x\} \subseteq y$ and $x \in y$. It involves both term and formula definition.

```
from graph import *
```

```
deff ("C", "Ax>exaexb")
deft ("I", "{x=xa")
start ("AxAyX:a:axI:byCexy")

"""

Line number 0



----



0.  (Ax : (Ay : ((I(a:x) C y) == x e y)))

by lines [-1]
Next!"""

right()

"""

Line number 1



----



0.  (Ay : ((I(a:x'!) C y) == x'! e y))

by lines [-1]
Next!"""

right()

"""
```

```
Line number 2


----


0.  ((I(a:x'!) C y'!) == x'! e y'!)

by lines [-1]
Next!"""

right()

"""

Line number 3



0.  (I(a:x'!) C y'!)
----


0.  x'! e y'!

by lines [-1]
Next!"""

left()

"""

Line number 5
```

```
0.  (A@x : (@x e I(a:x'!) -> @x e y'!))
----



0.  x'! e y'!

by lines [-1]
Next!"""

left()

"""

Line number 6



0.  (x*? e I(a:x'!) -> x*? e y'!)

1.  (A@x : (@x e I(a:x'!) -> @x e y'!))
----



0.  x'! e y'!

by lines [-1]
Next!"""

left()

"""

Line number 7
```

```
0.  (A@x : (@x e I(a:x'!) -> @x e y'!))
----



0.  x*? e I(a:x'!)

1.  x'! e y'!

by lines [-1]
Next!"""

right()

"""

Line number 9



0.  (A@x : (@x e I(a:x'!) -> @x e y'!))
----



0.  x*? e {@x | @x = x'!}

1.  x'! e y'!

by lines [-1]
Next!"""

right()

"""

Line number 10
```

```
0.  (A@x : (@x e I(a:x'!) -> @x e y'!))
----



0.  x*? = x'!

1.  x'! e y'!

by lines [-1]
Next!"""

right()

"""

Line number 11



0.  (A@x : (@x e I(a:x'!) -> @x e y'!))
----



0.  (Ax'' : (x'' e x*? == x'' e x'!))

1.  x'! e y'!

by lines [-1]
Next!"""

right()

"""
```

```
Line number 12



0.  (A@x : (@x e I(a:x'!) -> @x e y'!))
----



0.  (x'*! e x*? == x'*! e x'!)

1.  x'! e y'!

by lines [-1]
Next!"""

right()

"""

Line number 13



0.  x'*! e x*?

1.  (A@x : (@x e I(a:x'!) -> @x e y'!))
----



0.  x'*! e x'!

1.  x'! e y'!

by lines [-1]
Next!"""
```

```
done()

"""

Line number 14



0.  x'*! e x'!

1.  (A@x : (@x e I(a:x'!) -> @x e y'!))
----



0.  x'*! e x'!

1.  x'! e y'!

by lines [-1]
Next!"""

done()

"""

Line number 8



0.  x'! e y'!

1.  (A@x : (@x e I(a:x'!) -> @x e y'!))
----



0.  x'! e y'!
```

```
by lines [-1]
Next!"""

done()

"""

Line number 4




0.  x'! e y'!
----



0.  (I(a:x'!) C y'!)

by lines [-1]
Next!"""

right()

"""

Line number 15




0.  x'! e y'!
----



0.  (A@x : (@x e I(a:x'!) -> @x e y'!))

by lines [-1]
```

```
Next!"""

right()

"""

Line number 16



0.  x'! e y'!
----



0.  (x*'! e I(a:x'!) -> x*'! e y'!)

by lines [-1]
Next!"""

right()

"""

Line number 17



0.  x*'! e I(a:x'!)

1.  x'! e y'!
----



0.  x*'! e y'!

by lines [-1]
```

```
Next!"""

left()

"""

Line number 18



0.  x*'! e {@x | @x = x'!}

1.  x'! e y'!
----



0.  x*'! e y'!

by lines [-1]
Next!"""

left()

"""

Line number 19



0.  x*'! = x'!

1.  x'! e y'!
----



0.  x*'! e y'!
```

```
by lines [-1]
Next!"""

left()

"""

Line number 20




0.  (Ax** : (x*'! e x** == x'! e x**))

1.  x'! e y'!
----



0.  x*'! e y'!

by lines [-1]
Next!"""

left()

"""

Line number 21




0.  (x*'! e x'''? == x'! e x'''?)

1.  (Ax** : (x*'! e x** == x'! e x**))

2.  x'! e y'!
----
```

```
0.  x*'! e y'!

by lines [-1]
Next!"""

left()

"""

Line number 22



0.  (x*'! e x'''? -> x'! e x'''?)

1.  (x'! e x'''? -> x*'! e x'''?)

2.  (Ax** : (x*'! e x** == x'! e x**))

3.  x'! e y'!
----



0.  x*'! e y'!

by lines [-1]
Next!"""

getleft(1)

"""

Line number 22
```

```
0.  (x'! e x'''? -> x*'! e x'''?)

1.  (Ax** : (x*'! e x** == x'! e x**))

2.  x'! e y'!

3.  (x*'! e x'''? -> x'! e x'''?)
----



0.  x*'! e y'!

by lines [-1]
Next!"""

left()

"""

Line number 23



0.  (Ax** : (x*'! e x** == x'! e x**))

1.  x'! e y'!

2.  (x*'! e x'''? -> x'! e x'''?)
----



0.  x'! e x'''?

1.  x*'! e y'!
```

```
by lines [-1]
Next!"""

getleft(1)

"""

Line number 23




0.  x'! e y'!

1.  (x*'! e x'''? -> x'! e x'''?)

2.  (Ax** : (x*'! e x** == x'! e x**))
----



0.  x'! e x'''?

1.  x*'! e y'!

by lines [-1]
Next!"""

done()

"""

Line number 24




0.  x*'! e y'!

1.  (Ax** : (x*'! e x** == x'! e x**))
```

```
2.  x'! e y'!

3.  (x*'! e y'! -> x'! e y'!)
----



0.  x*'! e y'!

by lines [-1]
Next!"""

done()

"""Done!"""
```

7.6 First projection theorem for the ordered pair

There may be some wheel spinning in this proof. It contains definitions of the unordered pair, the ordered pair, and the first projection operator on ordered pairs, and the proof that the first projection operator works.

```
from graph import *
setlog ("pairs3")
deft ("P", "{xV=xa=xb")
deft ("O", ":a:aa:baP:b:aa:bbPP")
deft ("'P", "{xAy>eyaexy")
start ("AxAy=:a:ax:byO'P{u=xu")

"""

Line number 0



----
```

```
0.  (Ax : (Ay : P'(a:(x O y)) = {u | x = u}))

by lines [-1]
Next!"""

right()

"""

Line number 1



----



0.  (Ay : P'(a:(x'! O y)) = {u | x'! = u})

by lines [-1]
Next!"""

right()

"""

Line number 2



----



0.  P'(a:(x'! O y'!)) = {u | x'! = u}

by lines [-1]
Next!"""
```

```
right()

"""

Line number 3



----



0.  {@x | (A@y : (@y e (x'! O y'!) -> @x e @y))} = {u | x'! = u}

by lines [-1]
Next!"""

right()

"""

Line number 4



----



0.  (Ax* : (x* e {@x | (A@y : (@y e (x'! O y'!) -> @x e @y))} == x* e {u | x'! = u}))

by lines [-1]
Next!"""

right()

"""

Line number 5
```

```
----


0.  (x''! e {@x | (A@y : (@y e (x'! O y'!) -> @x e @y))} == x''! e {u | x'! = u})

by lines [-1]
Next!"""

right()

"""

Line number 6




0.  x''! e {@x | (A@y : (@y e (x'! O y'!) -> @x e @y))}
----



0.  x''! e {u | x'! = u}

by lines [-1]
Next!"""

right()

"""

Line number 8




0.  x''! e {@x | (A@y : (@y e (x'! O y'!) -> @x e @y))}
```

```
----



0.  x'! = x''!

by lines [-1]
Next!"""

left()

"""

Line number 9



0.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  x'! = x''!

by lines [-1]
Next!"""

left()

"""

Line number 10



0.  (y*? e (x'! O y'!) -> x''! e y*?)

1.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
```

```
----



0.  x'! = x''!

by lines [-1]
Next!"""

left()

"""

Line number 11




0.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  y*? e (x'! O y'!)

1.  x'! = x''!

by lines [-1]
Next!"""

right()

"""

Line number 13




0.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
```

```
----



0.  y*? e ((x'! P x'!) P (x'! P y'!))

1.  x'! = x''!

by lines [-1]
Next!"""

right()

"""

Line number 14



0.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  y*? e {@x | (@x = (x'! P x'!) V @x = (x'! P y'!))}

1.  x'! = x''!

by lines [-1]
Next!"""

right()

"""

Line number 15
```

```
0.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  (y*? = (x'! P x'!) V y*? = (x'! P y'!))

1.  x'! = x''!

by lines [-1]
Next!"""

right()

"""

Line number 16



0.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  y*? = (x'! P x'!)

1.  y*? = (x'! P y'!)

2.  x'! = x''!

by lines [-1]
Next!"""

setunknown ("y*",":a'x:b'xP")

"""
```

```
Line number 16



0.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  (x'! P x'!) = (x'! P x'!)

1.  (x'! P x'!) = (x'! P y'!)

2.  x'! = x''!

by lines [-1]
Next!"""

right()

"""

Line number 17



0.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  (x'! P x'!) = {@x | (@x = x'! V @x = x'!)}

1.  (x'! P x'!) = (x'! P y'!)

2.  x'! = x''!

by lines [-1]
```

```
Next!"""

right()

"""

Line number 18



0.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  {@x | (@x = x'! V @x = x'!)} = {@x | (@x = x'! V @x = x'!)}

1.  (x'! P x'!) = (x'! P y'!)

2.  x'! = x''!

by lines [-1]
Next!"""

right()

"""

Line number 19



0.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  (Ax'* : (x'* e {@x | (@x = x'! V @x = x'!)} == x'* e {@x | (@x = x'! V @x = x'!)}))
```

```
1.  (x'! P x'!) = (x'! P y'!)

2.  x'! = x''!

by lines [-1]
Next!"""

right()

"""

Line number 20




0.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  (x*'! e {@x | (@x = x'! V @x = x'!)} == x*'! e {@x | (@x = x'! V @x = x'!)})

1.  (x'! P x'!) = (x'! P y'!)

2.  x'! = x''!

by lines [-1]
Next!"""

right()

"""

Line number 21
```

```
0.  x*'! e {@x | (@x = x'! V @x = x'!)}

1.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  x*'! e {@x | (@x = x'! V @x = x'!)}

1.  (x'! P x'!) = (x'! P y'!)

2.  x'! = x''!

by lines [-1]
Next!"""

done()

"""

Line number 22



0.  x*'! e {@x | (@x = x'! V @x = x'!)}

1.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  x*'! e {@x | (@x = x'! V @x = x'!)}

1.  (x'! P x'!) = (x'! P y'!)

2.  x'! = x''!

by lines [-1]
Next!"""
```

```
done()

"""

Line number 12



0.  x''! e (x'! P x'!)

1.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  x'! = x''!

by lines [-1]
Next!"""

left()

"""

Line number 23



0.  x''! e {@x | (@x = x'! V @x = x'!)}

1.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  x'! = x''!
```

```
by lines [-1]
Next!"""

left()

"""

Line number 24




0.  (x''! = x'! V x''! = x'!)

1.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  x'! = x''!

by lines [-1]
Next!"""

left()

"""

Line number 25




0.  x''! = x'!

1.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----
```

```
0.  x'! = x''!

by lines [-1]
Next!"""

done()
left()

"""

Line number 27



0.  (Ax''* : (x''! e x''* == x'! e x''*))

1.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  x'! = x''!

by lines [-1]
Next!"""

left()

"""

Line number 28



0.  (x''! e x'*'? == x'! e x'*'?)

1.  (Ax''* : (x''! e x''* == x'! e x''*))
```

```
2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  x'! = x''!

by lines [-1]
Next!"""

setunknown ("x'*'","{u='xu")

"""

Line number 28



0.  (x''! e {u | x'! = u} == x'! e {u | x'! = u})

1.  (Ax''* : (x''! e x''* == x'! e x''*))

2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  x'! = x''!

by lines [-1]
Next!"""

left()

"""

Line number 29
```

```
0.  (x''! e {u | x'! = u} -> x'! e {u | x'! = u})

1.  (x'! e {u | x'! = u} -> x''! e {u | x'! = u})

2.  (Ax''* : (x''! e x''* == x'! e x''*))

3.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  x'! = x''!

by lines [-1]
Next!"""

getleft(1)

"""

Line number 29



0.  (x'! e {u | x'! = u} -> x''! e {u | x'! = u})

1.  (Ax''* : (x''! e x''* == x'! e x''*))

2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

3.  (x''! e {u | x'! = u} -> x'! e {u | x'! = u})
----



0.  x'! = x''!
```

```
by lines [-1]
Next!"""

left()

"""

Line number 30




0.  (Ax''* : (x''! e x''* == x'! e x''*))

1.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

2.  (x''! e {u | x'! = u} -> x'! e {u | x'! = u})
----



0.  x'! e {u | x'! = u}

1.  x'! = x''!

by lines [-1]
Next!"""

right()

"""

Line number 32




0.  (Ax''* : (x''! e x''* == x'! e x''*))

1.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
```

```
2.  (x''! e {u | x'! = u} -> x'! e {u | x'! = u})
----



0.  x'! = x'!

1.  x'! = x''!

by lines [-1]
Next!"""

right()

"""

Line number 33



0.  (Ax''* : (x''! e x''* == x'! e x''*))

1.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

2.  (x''! e {u | x'! = u} -> x'! e {u | x'! = u})
----



0.  (Ax'** : (x'** e x'! == x'** e x'!))

1.  x'! = x''!

by lines [-1]
Next!"""

right()
```

```
"""

Line number 34



0.  (Ax''* : (x''! e x''* == x'! e x''*))

1.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

2.  (x''! e {u | x'! = u} -> x'! e {u | x'! = u})
----



0.  (x*''! e x'! == x*''! e x'!)

1.  x'! = x''!

by lines [-1]
Next!"""

right()

"""

Line number 35



0.  x*''! e x'!

1.  (Ax''* : (x''! e x''* == x'! e x''*))

2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

3.  (x''! e {u | x'! = u} -> x'! e {u | x'! = u})
----
```

```
0.  x*''! e x'!

1.  x'! = x''!

by lines [-1]
Next!"""

done()

"""

Line number 36



0.  x*''! e x'!

1.  (Ax''* : (x''! e x''* == x'! e x''*))

2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

3.  (x''! e {u | x'! = u} -> x'! e {u | x'! = u})
----



0.  x*''! e x'!

1.  x'! = x''!

by lines [-1]
Next!"""

done()

"""
```

```
Line number 31



0.  x''! e {u | x'! = u}

1.  (Ax''* : (x''! e x''* == x'! e x''*))

2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

3.  (x''! e {u | x'! = u} -> x'! e {u | x'! = u})
----



0.  x'! = x''!

by lines [-1]
Next!"""

left()

"""

Line number 37



0.  x'! = x''!

1.  (Ax''* : (x''! e x''* == x'! e x''*))

2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

3.  (x''! e {u | x'! = u} -> x'! e {u | x'! = u})
----
```

```
0.  x'! = x''!

by lines [-1]
Next!"""

done()

"""

Line number 26




0.  x''! = x'!

1.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  x'! = x''!

by lines [-1]
Next!"""

left()

"""

Line number 38




0.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

1.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
```

```
----



0.  x'! = x''!

by lines [-1]
Next!"""

left()

"""

Line number 39




0.  (x''! e x**'? == x'! e x**'?)

1.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  x'! = x''!

by lines [-1]
Next!"""

#setunknown ("x'*'","{u-'xu")
setunknown ("x'*'","{u='xu")

"""

Line number 39
```

```
0.  (x''! e x**'? == x'! e x**'?)

1.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  x'! = x''!

by lines [-1]
Next!"""

right()

"""

Line number 40



0.  (x''! e x**'? == x'! e x**'?)

1.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  (Ax*** : (x*** e x'! == x*** e x''!))

by lines [-1]
Next!"""

right()
```

```
"""

Line number 41



0.  (x''! e x**'? == x'! e x**'?)

1.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  (x''''! e x'! == x''''! e x''!)

by lines [-1]
Next!"""

right()

"""

Line number 42



0.  x''''! e x'!

1.  (x''! e x**'? == x'! e x**'?)

2.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

3.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----
```

```
0.  x’’’’! e x’’!

by lines [-1]
Next!"""

right()

"""

Line number 42




0.  x’’’’! e x’!

1.  (x’’! e x**’? == x’! e x**’?)

2.  (Ax*’* : (x’’! e x*’* == x’! e x*’*))

3.  (A@y : (@y e (x’! O y’!) -> x’’! e @y))
----



0.  x’’’’! e x’’!

by lines [-1]
Next!"""

done()
getleft(1)

"""

Line number 42
```

```
0.  (x''! e x**'? == x'! e x**'?)

1.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

3.  x''''! e x'!
----



0.  x''''! e x''!

by lines [-1]
Next!"""

setunknown ("x**'",{ue'''xu")

"""

Line number 42



0.  (x''! e x**'? == x'! e x**'?)

1.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

3.  x''''! e x'!
----



0.  x''''! e x''!
```

```
by lines [-1]
Next!"""

getleft(1)

"""

Line number 42



0.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

1.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

2.  x''''! e x'!

3.  (x''! e x**'? == x'! e x**'?)
----



0.  x''''! e x''!

by lines [-1]
Next!"""

left()

"""

Line number 44



0.  (x''! e x'''*? == x'! e x'''*?)

1.  (Ax*'* : (x''! e x*'* == x'! e x*'*))
```

```
2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

3.  x''''! e x'!

4.  (x''! e x**'? == x'! e x**'?)
----



0.  x''''! e x''!

by lines [-1]
Next!"""

setunknown ("x'''*","{ue''''xu")

"""

Line number 44



0.  (x''! e {u | x''''! e u} == x'! e {u | x''''! e u})

1.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

3.  x''''! e x'!

4.  (x''! e x**'? == x'! e x**'?)
----



0.  x''''! e x''!

by lines [-1]
```

```
Next!"""

left()

"""

Line number 45



0.  (x''! e {u | x''''! e u} -> x'! e {u | x''''! e u})

1.  (x'! e {u | x''''! e u} -> x''! e {u | x''''! e u})

2.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

3.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

4.  x''''! e x'!

5.  (x''! e x**'? == x'! e x**'?)
----



0.  x''''! e x''!

by lines [-1]
Next!"""

left()

"""

Line number 46
```

```
0.  (x'! e {u | x''''! e u} -> x''! e {u | x''''! e u})

1.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

3.  x''''! e x'!

4.  (x''! e x**'? == x'! e x**'?)
----



0.  x''! e {u | x''''! e u}

1.  x''''! e x''!

by lines [-1]
Next!"""

right()

"""

Line number 48



0.  (x'! e {u | x''''! e u} -> x''! e {u | x''''! e u})

1.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

3.  x''''! e x'!

4.  (x''! e x**'? == x'! e x**'?)
----
```

```
0.  x’’’’! e x’’!

1.  x’’’’! e x’’!

by lines [-1]
Next!"""

left()

"""

Line number 49



0.  (Ax*’* : (x’’! e x*’* == x’! e x*’*))

1.  (A@y : (@y e (x’! O y’!) -> x’’! e @y))

2.  x’’’’! e x’!

3.  (x’’! e x**’? == x’! e x**’?)
----



0.  x’! e {u | x’’’’! e u}

1.  x’’’’! e x’’!

2.  x’’’’! e x’’!

by lines [-1]
Next!"""

right()
```

```
"""

Line number 51



0.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

1.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

2.  x''''! e x'!

3.  (x''! e x**'? == x'! e x**'?)
----



0.  x''''! e x'!

1.  x''''! e x''!

2.  x''''! e x''!

by lines [-1]
Next!"""

getleft(2)

"""

Line number 51



0.  x''''! e x'!

1.  (x''! e x**'? == x'! e x**'?)
```

```
2.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

3.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  x''''! e x'!

1.  x''''! e x''!

2.  x''''! e x''!

by lines [-1]
Next!"""

done()

"""

Line number 50



0.  x''! e {u | x''''! e u}

1.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

3.  x''''! e x'!

4.  (x''! e x**'? == x'! e x**'?)
----



0.  x''''! e x''!
```

```
1.  x’’’’! e x’’!

by lines [-1]
Next!"""

left()

"""

Line number 52




0.  x’’’’! e x’’!

1.  (Ax*’* : (x’’! e x*’* == x’! e x*’*))

2.  (A@y : (@y e (x’! O y’!) -> x’’! e @y))

3.  x’’’’! e x’!

4.  (x’’! e x**’? == x’! e x**’?)
----



0.  x’’’’! e x’’!

1.  x’’’’! e x’’!

by lines [-1]
Next!"""

done()

"""

Line number 47
```

```
0.  x’! e {u | x’’’’! e u}

1.  (x’! e {u | x’’’’! e u} -> x’’! e {u | x’’’’! e u})

2.  (Ax*’* : (x’’! e x*’* == x’! e x*’*))

3.  (A@y : (@y e (x’! O y’!) -> x’’! e @y))

4.  x’’’’! e x’!

5.  (x’’! e x**’? == x’! e x**’?)
----



0.  x’’’’! e x’’!

by lines [-1]
Next!"""

left()

"""

Line number 53



0.  x’’’’! e x’!

1.  (x’! e {u | x’’’’! e u} -> x’’! e {u | x’’’’! e u})

2.  (Ax*’* : (x’’! e x*’* == x’! e x*’*))

3.  (A@y : (@y e (x’! O y’!) -> x’’! e @y))
```

```
4.  x’’’’! e x’!

5.  (x’’! e x**’? == x’! e x**’?)
----



0.  x’’’’! e x’’!

by lines [-1]
Next!"""

done()
getleft(1)

"""

Line number 53



0.  (x’! e {u | x’’’’! e u} -> x’’! e {u | x’’’’! e u})

1.  (Ax*’* : (x’’! e x*’* == x’! e x*’*))

2.  (A@y : (@y e (x’! O y’!) -> x’’! e @y))

3.  x’’’’! e x’!

4.  (x’’! e x**’? == x’! e x**’?)

5.  x’’’’! e x’!
----



0.  x’’’’! e x’’!

by lines [-1]
```

```
Next!"""

left()

"""

Line number 54



0.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

1.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

2.  x''''! e x'!

3.  (x''! e x**'? == x'! e x**'?)

4.  x''''! e x'!
----



0.  x'! e {u | x''''! e u}

1.  x''''! e x''!

by lines [-1]
Next!"""

right()

"""

Line number 56
```

```
0.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

1.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

2.  x''''! e x'!

3.  (x''! e x**'? == x'! e x**'?)

4.  x''''! e x'!
----



0.  x''''! e x'!

1.  x''''! e x''!

by lines [-1]
Next!"""

getleft(2)

"""

Line number 56




0.  x''''! e x'!

1.  (x''! e x**'? == x'! e x**'?)

2.  x''''! e x'!

3.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

4.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----
```

```
0.  x’’’’! e x’!

1.  x’’’’! e x’’!

by lines [-1]
Next!"""

done()

"""

Line number 55



0.  x’’! e {u | x’’’’! e u}

1.  (Ax*’* : (x’’! e x*’* == x’! e x*’*))

2.  (A@y : (@y e (x’! O y’!) -> x’’! e @y))

3.  x’’’’! e x’!

4.  (x’’! e x**’? == x’! e x**’?)

5.  x’’’’! e x’!
----



0.  x’’’’! e x’’!

by lines [-1]
Next!"""

left()
```

```
"""

Line number 57



0.  x’’’’! e x’’!

1.  (Ax*’* : (x’’! e x*’* == x’! e x*’*))

2.  (A@y : (@y e (x’! O y’!) -> x’’! e @y))

3.  x’’’’! e x’!

4.  (x’’! e x**’? == x’! e x**’?)

5.  x’’’’! e x’!
----



0.  x’’’’! e x’’!

by lines [-1]
Next!"""

done()

"""

Line number 43



0.  x’’’’! e x’’!

1.  (x’’! e x**’? == x’! e x**’?)
```

```
2.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

3.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----



0.  x''''! e x'!

by lines [-1]
Next!"""

done()
getleft(2)

"""

Line number 43



0.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

1.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

2.  x''''! e x''!

3.  (x''! e x**'? == x'! e x**'?)
----



0.  x''''! e x'!

by lines [-1]
Next!"""

left()
```

```
"""

Line number 58



0.  (x''! e x''*'? == x'! e x''*'?)

1.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

3.  x''''! e x''!

4.  (x''! e x**'? == x'! e x**'?)
----



0.  x''''! e x'!

by lines [-1]
Next!"""

left()

"""

Line number 59



0.  (x''! e x''*'? -> x'! e x''*'?)

1.  (x'! e x''*'? -> x''! e x''*'?)

2.  (Ax*'* : (x''! e x*'* == x'! e x*'*))
```

```
3.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

4.  x''''! e x''!

5.  (x''! e x**'? == x'! e x**'?)
----



0.  x''''! e x'!

by lines [-1]
Next!"""

setunknown ("x''*'","{ue'''xu")

"""

Line number 59



0.  (x''! e {u | x''''! e u} -> x'! e {u | x''''! e u})

1.  (x'! e {u | x''''! e u} -> x''! e {u | x''''! e u})

2.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

3.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

4.  x''''! e x''!

5.  (x''! e x**'? == x'! e x**'?)
----



0.  x''''! e x'!
```

```
by lines [-1]
Next!"""

left()

"""

Line number 60



0.  (x'! e {u | x''''! e u} -> x''! e {u | x''''! e u})

1.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

3.  x''''! e x''!

4.  (x''! e x**'? == x'! e x**'?)
----



0.  x''! e {u | x''''! e u}

1.  x''''! e x'!

by lines [-1]
Next!"""

right()

"""

Line number 62
```

```
0.  (x'! e {u | x''''! e u} -> x''! e {u | x''''! e u})

1.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

2.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

3.  x''''! e x''!

4.  (x''! e x**'? == x'! e x**'?)
----



0.  x''''! e x''!

1.  x''''! e x'!

by lines [-1]
Next!"""

getleft(3)

"""

Line number 62



0.  x''''! e x''!

1.  (x''! e x**'? == x'! e x**'?)

2.  (x'! e {u | x''''! e u} -> x''! e {u | x''''! e u})

3.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

4.  (A@y : (@y e (x'! O y'!) -> x''! e @y))
----
```

```
0.  x’’’’! e x’’!

1.  x’’’’! e x’!

by lines [-1]
Next!"""

done()

"""

Line number 61



0.  x’! e {u | x’’’’! e u}

1.  (x’! e {u | x’’’’! e u} -> x’’! e {u | x’’’’! e u})

2.  (Ax*’* : (x’’! e x*’* == x’! e x*’*))

3.  (A@y : (@y e (x’! O y’!) -> x’’! e @y))

4.  x’’’’! e x’’!

5.  (x’’! e x**’? == x’! e x**’?)
----



0.  x’’’’! e x’!

by lines [-1]
Next!"""

left()
```

```
"""

Line number 63



0.  x''''! e x'!

1.  (x'! e {u | x''''! e u} -> x''! e {u | x''''! e u})

2.  (Ax*'* : (x''! e x*'* == x'! e x*'*))

3.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

4.  x''''! e x''!

5.  (x''! e x**'? == x'! e x**'?)
----



0.  x''''! e x'!

by lines [-1]
Next!"""

done()

"""

Line number 7



0.  x''! e {u | x'! = u}
----
```

```
0.  x''! e {@x | (A@y : (@y e (x'! O y'!) -> @x e @y))}

by lines [-1]
Next!"""

left()

"""

Line number 64




0.  x'! = x''!
----



0.  x''! e {@x | (A@y : (@y e (x'! O y'!) -> @x e @y))}

by lines [-1]
Next!"""

right()

"""

Line number 65




0.  x'! = x''!
----
```

```
0.  (A@y : (@y e (x'! O y'!) -> x''! e @y))

by lines [-1]
Next!"""

right()

"""

Line number 66




0.  x'! = x''!
----



0.  (y''! e (x'! O y'!) -> x''! e y''!)

by lines [-1]
Next!"""

right()

"""

Line number 67




0.  y''! e (x'! O y'!)

1.  x'! = x''!
----
```

```
0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 68



0.  y''! e ((x'! P x'!) P (x'! P y'!))

1.  x'! = x''!
----



0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 69



0.  y''! e {@x | (@x = (x'! P x'!) V @x = (x'! P y'!))}

1.  x'! = x''!
----
```

```
0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 70



0.  (y''! = (x'! P x'!) V y''! = (x'! P y'!))

1.  x'! = x''!
----



0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 71



0.  y''! = (x'! P x'!)

1.  x'! = x''!
```

```
----



0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 73



0.  y''! = {@x | (@x = x'! V @x = x'!)}

1.  x'! = x''!
----



0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 74



0.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))
```

```
1.  x'! = x''!
----



0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 75



0.  (y''! e x'*''? == {@x | (@x = x'! V @x = x'!)} e x'*''?)

1.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

2.  x'! = x''!
----



0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 76
```

```
0.  (y''! e x'*''? -> {@x | (@x = x'! V @x = x'!)} e x'*''?)

1.  ({@x | (@x = x'! V @x = x'!)} e x'*''? -> y''! e x'*''?)

2.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

3.  x'! = x''!
----



0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 77



0.  ({@x | (@x = x'! V @x = x'!)} e x'*''? -> y''! e x'*''?)

1.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

2.  x'! = x''!
----



0.  y''! e x'*''?

1.  x''! e y''!
```

```
by lines [-1]
Next!"""

left()

"""

Line number 79



0.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

1.  x'! = x''!
----



0.  {@x | (@x = x'! V @x = x'!)} e x'*''?

1.  y''! e x'*''?

2.  x''! e y''!

by lines [-1]
Next!"""

right()

"""

Line number 79



0.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))
```

```
1.  x'! = x''!
----



0.  {@x | (@x = x'! V @x = x'!)} e x'*''?

1.  y''! e x'*''?

2.  x''! e y''!

by lines [-1]
Next!"""

back()
back()
back()
back()
back()
setunknown ("x'*''","{ue'xu")

"""

Line number 75



0.  (y''! e {u | x'! e u} == {@x | (@x = x'! V @x = x'!)} e {u | x'! e u})

1.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

2.  x'! = x''!
----



0.  x''! e y''!

by lines [-1]
```

```
Next!"""

left()

"""

Line number 76



0.  (y''! e {u | x'! e u} -> {@x | (@x = x'! V @x = x'!)} e {u | x'! e u})

1.  ({@x | (@x = x'! V @x = x'!)} e {u | x'! e u} -> y''! e {u | x'! e u})

2.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

3.  x'! = x''!
----



0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 77



0.  ({@x | (@x = x'! V @x = x'!)} e {u | x'! e u} -> y''! e {u | x'! e u})

1.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))
```

```
2.  x'! = x''!
----



0.  y''! e {u | x'! e u}

1.  x''! e y''!

by lines [-1]
Next!"""

right()

"""

Line number 79



0.  ({@x | (@x = x'! V @x = x'!)} e {u | x'! e u} -> y''! e {u | x'! e u})

1.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

2.  x'! = x''!
----



0.  x'! e y''!

1.  x''! e y''!

by lines [-1]
Next!"""

left()

"""
```

```
Line number 80



0.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

1.  x'! = x''!
----



0.  {@x | (@x = x'! V @x = x'!)} e {u | x'! e u}

1.  x'! e y''!

2.  x''! e y''!

by lines [-1]
Next!"""

right()

"""

Line number 82



0.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

1.  x'! = x''!
----



0.  x'! e {@x | (@x = x'! V @x = x'!)}
```

```
1.  x'! e y''!

2.  x''! e y''!

by lines [-1]
Next!"""

right()

"""

Line number 83



0.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

1.  x'! = x''!
----



0.  (x'! = x'! V x'! = x'!)

1.  x'! e y''!

2.  x''! e y''!

by lines [-1]
Next!"""

right()

"""

Line number 84
```

```
0.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

1.  x'! = x''!
----



0.  x'! = x'!

1.  x'! = x'!

2.  x'! e y''!

3.  x''! e y''!

by lines [-1]
Next!"""

right()

"""

Line number 85



0.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

1.  x'! = x''!
----



0.  (Ax'*'* : (x'*'* e x'! == x'*'* e x'!))

1.  x'! = x'!

2.  x'! e y''!
```

```
3.  x''! e y''!

by lines [-1]
Next!"""

right()

"""

Line number 86



0.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

1.  x'! = x''!
----



0.  (x'**'! e x'! == x'**'! e x'!)

1.  x'! = x'!

2.  x'! e y''!

3.  x''! e y''!

by lines [-1]
Next!"""

right()

"""

Line number 87
```

```
0.  x'**'! e x'!

1.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

2.  x'! = x''!
----



0.  x'**'! e x'!

1.  x'! = x'!

2.  x'! e y''!

3.  x''! e y''!

by lines [-1]
Next!"""

done()

"""

Line number 88



0.  x'**'! e x'!

1.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

2.  x'! = x''!
----
```

```
0.  x'**'! e x'!

1.  x'! = x'!

2.  x'! e y''!

3.  x''! e y''!

by lines [-1]
Next!"""

done()

"""

Line number 81



0.  y''! e {u | x'! e u}

1.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

2.  x'! = x''!
----



0.  x'! e y''!

1.  x''! e y''!

by lines [-1]
Next!"""

left()

"""
```

```
Line number 89



0.  x'! e y''!

1.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

2.  x'! = x''!
----



0.  x'! e y''!

1.  x''! e y''!

by lines [-1]
Next!"""

done()

"""

Line number 78



0.  {@x | (@x = x'! V @x = x'!)} e {u | x'! e u}

1.  ({@x | (@x = x'! V @x = x'!)} e {u | x'! e u} -> y''! e {u | x'! e u})

2.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

3.  x'! = x''!
----
```

```
0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 90



0.  x'! e {@x | (@x = x'! V @x = x'!)}

1.  ({@x | (@x = x'! V @x = x'!)} e {u | x'! e u} -> y''! e {u | x'! e u})

2.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

3.  x'! = x''!
----



0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 91
```

```
0.  (x'! = x'! V x'! = x'!)

1.  ({@x | (@x = x'! V @x = x'!)} e {u | x'! e u} -> y''! e {u | x'! e u})

2.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

3.  x'! = x''!
----



0.  x''! e y''!

by lines [-1]
Next!"""

getleft(1)

"""

Line number 91



0.  ({@x | (@x = x'! V @x = x'!)} e {u | x'! e u} -> y''! e {u | x'! e u})

1.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

2.  x'! = x''!

3.  (x'! = x'! V x'! = x'!)
----



0.  x''! e y''!

by lines [-1]
Next!"""
```

```
left()

"""

Line number 92



0.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

1.  x'! = x''!

2.  (x'! = x'! V x'! = x'!)
----



0.  {@x | (@x = x'! V @x = x'!)} e {u | x'! e u}

1.  x''! e y''!

by lines [-1]
Next!"""

right()

"""

Line number 94



0.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

1.  x'! = x''!

2.  (x'! = x'! V x'! = x'!)
```

```
----



0.  x'! e {@x | (@x = x'! V @x = x'!)}

1.  x''! e y''!

by lines [-1]
Next!"""

right()

"""

Line number 95



0.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

1.  x'! = x''!

2.  (x'! = x'! V x'! = x'!)
----



0.  (x'! = x'! V x'! = x'!)

1.  x''! e y''!

by lines [-1]
Next!"""

right()

"""
```

```
Line number 96



0.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

1.  x'! = x''!

2.  (x'! = x'! V x'! = x'!)
----



0.  x'! = x'!

1.  x'! = x'!

2.  x''! e y''!

by lines [-1]
Next!"""

right()

"""

Line number 97



0.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

1.  x'! = x''!

2.  (x'! = x'! V x'! = x'!)
----
```

```
0.  (Ax'*** : (x'*** e x'! == x'*** e x'!))

1.  x'! = x'!

2.  x''! e y''!

by lines [-1]
Next!"""

right()

"""

Line number 98



0.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

1.  x'! = x''!

2.  (x'! = x'! V x'! = x'!)
----



0.  (x*'''! e x'! == x*'''! e x'!)

1.  x'! = x'!

2.  x''! e y''!

by lines [-1]
Next!"""

right()

"""
```

```
Line number 99



0.  x*’’’! e x’!

1.  (Ax’’** : (y’’! e x’’** == {@x | (@x = x’! V @x = x’!)} e x’’**))

2.  x’! = x’’!

3.  (x’! = x’! V x’! = x’!)
----



0.  x*’’’! e x’!

1.  x’! = x’!

2.  x’’! e y’’!

by lines [-1]
Next!"""

done()

"""

Line number 100



0.  x*’’’! e x’!

1.  (Ax’’** : (y’’! e x’’** == {@x | (@x = x’! V @x = x’!)} e x’’**))

2.  x’! = x’’!
```

```
3.  (x'! = x'! V x'! = x'!)
----



0.  x*'''! e x'!

1.  x'! = x'!

2.  x''! e y''!

by lines [-1]
Next!"""

done()

"""

Line number 93



0.  y''! e {u | x'! e u}

1.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

2.  x'! = x''!

3.  (x'! = x'! V x'! = x'!)
----



0.  x''! e y''!

by lines [-1]
Next!"""
```

```
left()

"""

Line number 101



0.  x'! e y''!

1.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))

2.  x'! = x''!

3.  (x'! = x'! V x'! = x'!)
----



0.  x''! e y''!

by lines [-1]
Next!"""

getleft(2)

"""

Line number 101



0.  x'! = x''!

1.  (x'! = x'! V x'! = x'!)

2.  x'! e y''!
```

```
3.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))
----



0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 102



0.  (Ax*''* : (x'! e x*''* == x''! e x*''*))

1.  (x'! = x'! V x'! = x'!)

2.  x'! e y''!

3.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))
----



0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 103
```

```
0.  (x'! e x*'*'? == x''! e x*'*'?)

1.  (Ax*''* : (x'! e x*''* == x''! e x*''*))

2.  (x'! = x'! V x'! = x'!)

3.  x'! e y''!

4.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))
----



0.  x''! e y''!

by lines [-1]
Next!"""

setunknown ("x*'*'","''y")

"""

Line number 103



0.  (x'! e y''! == x''! e y''!)

1.  (Ax*''* : (x'! e x*''* == x''! e x*''*))

2.  (x'! = x'! V x'! = x'!)

3.  x'! e y''!

4.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))
```

```
----



0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 104



0.  (x'! e y''! -> x''! e y''!)

1.  (x''! e y''! -> x'! e y''!)

2.  (Ax*''* : (x'! e x*''* == x''! e x*''*))

3.  (x'! = x'! V x'! = x'!)

4.  x'! e y''!

5.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))
----



0.  x''! e y''!

by lines [-1]
Next!"""

left()
```

```
"""

Line number 105



0.  (x''! e y''! -> x'! e y''!)

1.  (Ax*''* : (x'! e x*''* == x''! e x*''*))

2.  (x'! = x'! V x'! = x'!)

3.  x'! e y''!

4.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))
----



0.  x'! e y''!

1.  x''! e y''!

by lines [-1]
Next!"""

getleft(3)

"""

Line number 105



0.  x'! e y''!

1.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))
```

```
2.  (x''! e y''! -> x'! e y''!)

3.  (Ax*''* : (x'! e x*''* == x''! e x*''*))

4.  (x'! = x'! V x'! = x'!)
----



0.  x'! e y''!

1.  x''! e y''!

by lines [-1]
Next!"""

done()

"""

Line number 106



0.  x''! e y''!

1.  (x''! e y''! -> x'! e y''!)

2.  (Ax*''* : (x'! e x*''* == x''! e x*''*))

3.  (x'! = x'! V x'! = x'!)

4.  x'! e y''!

5.  (Ax''** : (y''! e x''** == {@x | (@x = x'! V @x = x'!)} e x''**))
----
```

```
0.  x''! e y''!

by lines [-1]
Next!"""

done()

"""

Line number 72




0.  y''! = (x'! P y'!)

1.  x'! = x''!
----



0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 107




0.  y''! = {@x | (@x = x'! V @x = y'!)}

1.  x'! = x''!
----
```

```
0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 108



0.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))

1.  x'! = x''!
----



0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 109



0.  (y''! e x**''? == {@x | (@x = x'! V @x = y'!)} e x**''?)

1.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))
```

```
2.  x'! = x''!
----



0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 110



0.  (y''! e x**''? -> {@x | (@x = x'! V @x = y'!)} e x**''?)

1.  ({@x | (@x = x'! V @x = y'!)} e x**''? -> y''! e x**''?)

2.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))

3.  x'! = x''!
----



0.  x''! e y''!

by lines [-1]
Next!"""

setunknown ("x**''","{ue''xu")

"""
```

```
Line number 110



0.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

1.  ({@x | (@x = x'! V @x = y'!)} e {u | x''! e u} -> y''! e {u | x''! e u})

2.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))

3.  x'! = x''!
----



0.  x''! e y''!

by lines [-1]
Next!"""

getleft(1)

"""

Line number 110



0.  ({@x | (@x = x'! V @x = y'!)} e {u | x''! e u} -> y''! e {u | x''! e u})

1.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))

2.  x'! = x''!

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})
----
```

```
0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 111



0.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))

1.  x'! = x''!

2.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})
----



0.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

1.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 113
```

```
0.  (y''! e x**'*? == {@x | (@x = x'! V @x = y'!)} e x**'*?)

1.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))

2.  x'! = x''!

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})
----



0.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

1.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 114



0.  (y''! e x**'*? -> {@x | (@x = x'! V @x = y'!)} e x**'*?)

1.  ({@x | (@x = x'! V @x = y'!)} e x**'*? -> y''! e x**'*?)

2.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))

3.  x'! = x''!

4.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})
----
```

```
0.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

1.  x''! e y''!

by lines [-1]
Next!"""

getleft(1)

"""

Line number 114



0.  ({@x | (@x = x'! V @x = y'!)} e x**'*? -> y''! e x**'*?)

1.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))

2.  x'! = x''!

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

4.  (y''! e x**'*? -> {@x | (@x = x'! V @x = y'!)} e x**'*?)
----



0.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

1.  x''! e y''!

by lines [-1]
Next!"""

left()

"""
```

```
Line number 115



0.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))

1.  x'! = x''!

2.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

3.  (y''! e x**'*? -> {@x | (@x = x'! V @x = y'!)} e x**'*?)
----



0.  {@x | (@x = x'! V @x = y'!)} e x**'*?

1.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

2.  x''! e y''!

by lines [-1]
Next!"""

right()

"""

Line number 115



0.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))

1.  x'! = x''!

2.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})
```

```
3.  (y''! e x**'*? -> {@x | (@x = x'! V @x = y'!)} e x**'*?)
----



0.  {@x | (@x = x'! V @x = y'!)} e x**'*?

1.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

2.  x''! e y''!

by lines [-1]
Next!"""

setunknown ("x**'*","{ue''xu")

"""

Line number 115



0.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))

1.  x'! = x''!

2.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})
----



0.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

1.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

2.  x''! e y''!
```

```
by lines [-1]
Next!"""

right()

"""

Line number 117



0.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))

1.  x'! = x''!

2.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})
----



0.  x''! e {@x | (@x = x'! V @x = y'!)}

1.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

2.  x''! e y''!

by lines [-1]
Next!"""

right()

"""

Line number 118
```

```
0.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))

1.  x'! = x''!

2.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})
----



0.  (x''! = x'! V x''! = y'!)

1.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

2.  x''! e y''!

by lines [-1]
Next!"""

right()

"""

Line number 119



0.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))

1.  x'! = x''!

2.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})
----
```

```
0.  x''! = x'!

1.  x''! = y'!

2.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

3.  x''! e y''!

by lines [-1]
Next!"""

getleft(1)

"""

Line number 119



0.  x'! = x''!

1.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

2.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

3.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))
----



0.  x''! = x'!

1.  x''! = y'!

2.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

3.  x''! e y''!
```

```
by lines [-1]
Next!"""

left()

"""

Line number 120



0.  (Ax***' : (x'! e x***' == x''! e x***'))

1.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

2.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

3.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))
----



0.  x''! = x'!

1.  x''! = y'!

2.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

3.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 121
```

```
0.  (x'! e x****? == x''! e x****?)

1.  (Ax***' : (x'! e x***' == x''! e x***'))

2.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

4.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))
----



0.  x''! = x'!

1.  x''! = y'!

2.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

3.  x''! e y''!

by lines [-1]
Next!"""

#setunknown ("x****","{u-u'x")
setunknown ("x****","{u=u'x")

"""

Line number 121



0.  (x'! e {u | u = x'!} == x''! e {u | u = x'!})

1.  (Ax***' : (x'! e x***' == x''! e x***'))
```

```
2.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

4.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))
----



0.  x''! = x'!

1.  x''! = y'!

2.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

3.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 122



0.  (x'! e {u | u = x'!} -> x''! e {u | u = x'!})

1.  (x''! e {u | u = x'!} -> x'! e {u | u = x'!})

2.  (Ax***' : (x'! e x***' == x''! e x***'))

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

4.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})
```

```
5.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))
----



0.  x''! = x'!

1.  x''! = y'!

2.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

3.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 123



0.  (x''! e {u | u = x'!} -> x'! e {u | u = x'!})

1.  (Ax***' : (x'! e x***' == x''! e x***'))

2.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

4.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))
----



0.  x'! e {u | u = x'!}
```

```
1.  x''! = x'!

2.  x''! = y'!

3.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

4.  x''! e y''!

by lines [-1]
Next!"""

right()

"""

Line number 125



0.  (x''! e {u | u = x'!} -> x'! e {u | u = x'!})

1.  (Ax***' : (x'! e x***' == x''! e x***'))

2.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

4.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))
----



0.  x'! = x'!

1.  x''! = x'!

2.  x''! = y'!

3.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}
```

```
4.  x''! e y''!

by lines [-1]
Next!"""

right()

"""

Line number 126



0.  (x''! e {u | u = x'!} -> x'! e {u | u = x'!})

1.  (Ax***' : (x'! e x***' == x''! e x***'))

2.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

4.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))
----



0.  (Ax''''' : (x''''' e x'! == x''''' e x'!))

1.  x''! = x'!

2.  x''! = y'!

3.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

4.  x''! e y''!

by lines [-1]
Next!"""
```

```
right()

"""

Line number 127



0.  (x''! e {u | u = x'!} -> x'! e {u | u = x'!})

1.  (Ax***' : (x'! e x***' == x''! e x***'))

2.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

4.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))
----



0.  (x''''*! e x'! == x''''*! e x'!)

1.  x''! = x'!

2.  x''! = y'!

3.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

4.  x''! e y''!

by lines [-1]
Next!"""

right()

"""
```

```
Line number 128



0.  x''''*! e x'!

1.  (x''! e {u | u = x'!} -> x'! e {u | u = x'!})

2.  (Ax***' : (x'! e x***' == x''! e x***'))

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

4.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

5.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))
----



0.  x''''*! e x'!

1.  x''! = x'!

2.  x''! = y'!

3.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

4.  x''! e y''!

by lines [-1]
Next!"""

done()

"""

Line number 129
```

```
0.  x''''*! e x'!

1.  (x''! e {u | u = x'!} -> x'! e {u | u = x'!})

2.  (Ax***' : (x'! e x***' == x''! e x***'))

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

4.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

5.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))
----



0.  x''''*! e x'!

1.  x''! = x'!

2.  x''! = y'!

3.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

4.  x''! e y''!

by lines [-1]
Next!"""

done()

"""

Line number 124



0.  x''! e {u | u = x'!}
```

```
1.  (x''! e {u | u = x'!} -> x'! e {u | u = x'!})

2.  (Ax***' : (x'! e x***' == x''! e x***'))

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

4.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

5.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))
----



0.  x''! = x'!

1.  x''! = y'!

2.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

3.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 130



0.  x''! = x'!

1.  (x''! e {u | u = x'!} -> x'! e {u | u = x'!})

2.  (Ax***' : (x'! e x***' == x''! e x***'))
```

```
3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

4.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

5.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))
----



0.  x''! = x'!

1.  x''! = y'!

2.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

3.  x''! e y''!

by lines [-1]
Next!"""

done()

"""

Line number 116



0.  y''! e {u | x''! e u}

1.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))

2.  x'! = x''!

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

4.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})
----
```

```
0.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

1.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 131




0.  x''! e y''!

1.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))

2.  x'! = x''!

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

4.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})
----



0.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

1.  x''! e y''!

by lines [-1]
Next!"""

getright(1)
```

```
"""

Line number 131



0.  x''! e y''!

1.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))

2.  x'! = x''!

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})

4.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})
----



0.  x''! e y''!

1.  {@x | (@x = x'! V @x = y'!)} e {u | x''! e u}

by lines [-1]
Next!"""

done()

"""

Line number 112



0.  y''! e {u | x''! e u}

1.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))
```

```
2.  x'! = x''!

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})
----



0.  x''! e y''!

by lines [-1]
Next!"""

left()

"""

Line number 132



0.  x''! e y''!

1.  (Ax*'** : (y''! e x*'** == {@x | (@x = x'! V @x = y'!)} e x*'**))

2.  x'! = x''!

3.  (y''! e {u | x''! e u} -> {@x | (@x = x'! V @x = y'!)} e {u | x''! e u})
----



0.  x''! e y''!

by lines [-1]
Next!"""

done()

"""Done!"""
```